



Hautes Études des Sciences et Techniques de  
l'Ingénierie et du Management

Rapport de stage

FILIÈRE : GÉNIE INFORMATIQUE ET INTELLIGENCE  
ARTIFICIELLE

---

CONCEPTION ET DÉVELOPPEMENT D'UNE  
PLATEFORME DE COMMANDE ET D'INTERACTION  
AUTONOME  
POUR UN ROBOT DE SERVICE ANDROID

---

*Réalisé par :*

Claudia LUSAMOTE  
KIMFUTA  
Ghassane KHALIFI

*Encadrante pédagogique :*

Mme MESKINI Fatime  
Zahra

Classe : 1<sup>ÈRE</sup> DU CYCLE D'INGÉNIEUR

Année universitaire : 2025 - 2026

## REMERCIEMENTS

Nous tenons à exprimer notre profonde gratitude à notre encadrante pédagogique, Mme MESKINI Fatime Zahra, pour son accompagnement, sa disponibilité et ses précieux conseils tout au long de la rédaction de ce rapport de stage.

Nous remercions également notre encadrant en entreprise, M. TULA Sri-dath, pour son expertise technique, ses orientations sur le robot CIOT et son soutien constant dans la conduite du projet au sein du laboratoire HESTIM.

Nous adressons nos sincères remerciements à l'ensemble du corps professoral ainsi qu'au personnel administratif et technique de HESTIM pour la qualité de la formation dispensée, leur professionnalisme et les moyens mis à notre disposition tout au long de notre parcours académique.

Nous remercions particulièrement les responsables de la filière Génie Informatique pour leur encadrement et les connaissances qu'ils nous ont transmises au cours de ces années d'études, nous permettant d'acquérir les compétences nécessaires à la réalisation de ce projet.

Nous souhaitons également exprimer notre reconnaissance envers toutes les personnes qui, de près ou de loin, ont contribué à la réussite de ce travail par leurs conseils, leurs encouragements ou leur soutien.

Enfin, nous adressons nos plus sincères remerciements à nos familles et à nos proches pour leur patience, leur confiance et leurs encouragements constants. Leur soutien moral a été une source de motivation essentielle tout au long de cette expérience.

## RÉSUMÉ

Les robots de service mobiles sont généralement livrés avec un écosystème logiciel fermé : application propriétaire, absence de documentation et de protocole de communication publié. Ce constat limite toute personnalisation par l'utilisateur final. Le projet CYBEL, mené dans le cadre d'un stage à HESTIM, vise à concevoir une plateforme de commande et d'interaction indépendante pour un robot de réception mobile CIOT TY1251D-03195, composé d'un châssis de navigation sous ROS et d'un « *upper body* » Android.

La démarche adoptée repose sur une rétro-ingénierie incrémentale et non destructive du protocole de communication interne du robot : balayage des services réseau exposés, introspection des topics et services ROS via **ros-bridge/rosapi**, et vérification systématique de l'effectivité de chaque commande avant de la considérer comme fonctionnelle. Sur cette base, une architecture en trois couches a été conçue :

- un SDK Python proposant une implémentation simulée (*mock*) et une implémentation réelle interchangeables ;
- une API FastAPI (REST + WebSocket) ;
- une interface web Vite/TypeScript opérateur ;

complétée par une interface visiteur kiosque (**frontend-kiosk/**) et deux applications Android natives légères (**CybelTTSBridge**, **CybelVisitorKiosk**).

À ce stade du stage, la connectivité avec le robot est établie, le protocole de télémétrie, de commande de vitesse et de navigation autonome a été reconstruit et intégré, la synthèse vocale (TTS) fonctionne via une application Android dédiée déclenchée par **am broadcast**, une interface opérateur complète (carte SLAM, LiDAR, visiteurs détectés, gestion du parcours de visite, arrêt d'urgence pendant une visite) est opérationnelle, et un déploiement embarqué sur Termux (*backend lite* + kiosque) est validé sur la tablette. L'interface visiteur a été recentrée sur une visite guidée autonome du laboratoire : huit arrêts synchronisés depuis **data/knowledgeV2-lab.json** vers **data/lab\_tour.json**, navigation par coordonnées (**/navi\_goal**) et annonces vocales séquentielles.

Ce rapport présente le contexte, la problématique, l'état de l'art, la conception, la méthodologie, l'implémentation réalisée, les résultats obtenus, ainsi qu'une analyse critique des limites et perspectives du projet.

**Mots-clés** : robotique de service, rétro-ingénierie de protocole, ROS, ros-bridge, FastAPI, interface homme-robot, navigation autonome, synthèse vocale, Termux, WebView Android.

## ABSTRACT

Mobile service robots are typically delivered with a closed software ecosystem : a proprietary application, no documentation, and no published communication protocol. This severely limits any customization by the end user. The CYBEL project, carried out as part of a final-year internship at HESTIM, aims to design an independent control and interaction platform for a CIOT TY1251D-03195 mobile reception robot, consisting of a ROS-based navigation chassis and an Android « *upper body* ».

The approach adopted relies on an incremental, non-destructive reverse engineering of the robot's internal communication protocol : scanning of exposed network services, introspection of ROS topics and services via **rosbridge/rosapi**, and systematic verification of the actual effect of each command before considering it functional. On this basis, a three-layer architecture was designed :

- a Python SDK offering interchangeable simulated (*mock*) and real implementations ;
- a FastAPI backend (REST + WebSocket) ;
- a Vite/TypeScript operator web interface ;

complemented by a visitor kiosk interface (**frontend-kiosk/**) and two lightweight native Android applications (**CybelTTSBridge**, **CybelVisitorKiosk**).

At this stage of the internship, connectivity with the robot has been established ; the telemetry, speed control, and autonomous navigation protocols have been reverse-engineered and integrated ; text-to-speech (TTS) works through a dedicated Android application triggered via **am broadcast** ; a complete operator interface (SLAM map, LiDAR, detected visitors, tour management, emergency stop during a visit) is operational ; and an embedded deployment on Termux (*backend lite* + kiosk) has been validated on the tablet. The visitor interface has been refocused on an autonomous guided tour of the laboratory : eight stops synchronized from **data/knowledgeV2-lab.json** to **data/lab\_tour.json**, coordinate-based navigation (**/navi\_goal**), and sequential voice announcements.

This report presents the context, problem statement, state of the art, design, methodology, implementation carried out, results obtained, as well as a critical analysis of the project's limitations and perspectives.

**Keywords** : service robotics, protocol reverse engineering, ROS, rosbridge, FastAPI, human–robot interface, autonomous navigation, text-to-speech, Termux, Android WebView.

# Table des matières

|                                                        |            |
|--------------------------------------------------------|------------|
| <b>REMERCIEMENTS</b>                                   | <b>i</b>   |
| <b>RÉSUMÉ</b>                                          | <b>ii</b>  |
| <b>ABSTRACT</b>                                        | <b>iii</b> |
| <b>LISTE DES ABRÉVIATIONS</b>                          | <b>x</b>   |
| <b>INTRODUCTION GÉNÉRALE</b>                           | <b>1</b>   |
| <b>1 Présentation de l'entreprise</b>                  | <b>2</b>   |
| 1.1 Activité principale et services . . . . .          | 2          |
| 1.2 Organisation interne . . . . .                     | 2          |
| 1.2.1 Organigramme de l'HESTIM . . . . .               | 2          |
| 1.3 Positionnement dans le secteur . . . . .           | 3          |
| 1.4 Valeurs et engagements . . . . .                   | 3          |
| 1.5 Lien avec notre stage . . . . .                    | 4          |
| 1.6 Conclusion . . . . .                               | 4          |
| <b>2 Analyse du besoin et étude préalable</b>          | <b>5</b>   |
| 2.1 Contexte du projet . . . . .                       | 5          |
| 2.2 Problématique . . . . .                            | 9          |
| 2.3 Objectifs du projet . . . . .                      | 10         |
| 2.3.1 Objectif général . . . . .                       | 10         |
| 2.3.2 Objectifs spécifiques . . . . .                  | 10         |
| 2.4 Cahier des charges . . . . .                       | 11         |
| 2.4.1 Périmètre du projet . . . . .                    | 11         |
| 2.4.2 Acteurs du système . . . . .                     | 11         |
| 2.4.3 Besoins fonctionnels . . . . .                   | 12         |
| 2.4.4 Besoins non fonctionnels . . . . .               | 13         |
| 2.4.5 Contraintes du projet . . . . .                  | 14         |
| 2.4.6 Livrables attendus . . . . .                     | 14         |
| 2.4.7 Critères de réception et de validation . . . . . | 14         |
| 2.5 Solution proposée . . . . .                        | 15         |
| 2.6 Conclusion . . . . .                               | 15         |
| <b>3 État de l'art</b>                                 | <b>16</b>  |
| 3.1 Fondements théoriques . . . . .                    | 16         |
| 3.2 État des solutions existantes . . . . .            | 16         |
| 3.3 Analyse comparative . . . . .                      | 16         |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| 3.4      | Limites des solutions existantes . . . . .                 | 16        |
| 3.5      | Conclusion . . . . .                                       | 17        |
| <b>4</b> | <b>Analyse et conception du système</b>                    | <b>19</b> |
| 4.1      | Analyse fonctionnelle . . . . .                            | 19        |
| 4.1.1    | Identification des acteurs . . . . .                       | 19        |
| 4.1.2    | Besoins fonctionnels . . . . .                             | 20        |
| 4.1.3    | Besoins non fonctionnels . . . . .                         | 21        |
| 4.2      | Architecture générale de CYBEL . . . . .                   | 22        |
| 4.3      | Architecture logicielle . . . . .                          | 22        |
| 4.3.1    | Couche d'accès au robot (SDK) . . . . .                    | 23        |
| 4.3.2    | Couche API . . . . .                                       | 23        |
| 4.3.3    | Couche présentation . . . . .                              | 24        |
| 4.4      | Diagramme des composants . . . . .                         | 24        |
| 4.5      | Diagrammes de cas d'utilisation . . . . .                  | 25        |
| 4.6      | Diagrammes de séquence . . . . .                           | 25        |
| 4.7      | Modèle des flux de communication . . . . .                 | 25        |
| 4.7.1    | Robot $\leftrightarrow$ rosbridge . . . . .                | 25        |
| 4.7.2    | rosbridge $\leftrightarrow$ Backend . . . . .              | 26        |
| 4.7.3    | Backend $\leftrightarrow$ Interfaces utilisateur . . . . . | 26        |
| 4.7.4    | Backend $\leftrightarrow$ Applications Android . . . . .   | 26        |
| 4.8      | Conclusion . . . . .                                       | 27        |
| <b>5</b> | <b>Méthodologie et planification</b>                       | <b>28</b> |
| 5.1      | Mode de travail et organisation . . . . .                  | 28        |
| 5.1.1    | Démarche de découverte protocolaire . . . . .              | 28        |
| 5.1.2    | Organisation hebdomadaire type . . . . .                   | 29        |
| 5.2      | Planning du projet . . . . .                               | 29        |
| 5.3      | Outils, langages et technologies utilisés . . . . .        | 31        |
| 5.3.1    | ROS (Robot Operating System) . . . . .                     | 31        |
| 5.3.2    | FastAPI . . . . .                                          | 32        |
| 5.3.3    | Python . . . . .                                           | 32        |
| 5.3.4    | TypeScript et Vite . . . . .                               | 32        |
| 5.3.5    | Android et Java . . . . .                                  | 33        |
| 5.3.6    | ADB (Android Debug Bridge) . . . . .                       | 33        |
| 5.3.7    | rosbridge et communication réseau . . . . .                | 34        |
| 5.4      | Découverte du Robot CIOT TY1251D-03195 « CYBEL » . . .     | 34        |
| 5.4.1    | Zenmmmap . . . . .                                         | 35        |
| 5.4.2    | JADX . . . . .                                             | 36        |
| 5.4.3    | Termux . . . . .                                           | 36        |
| 5.5      | Proposition de documentation technique . . . . .           | 37        |

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| 5.6      | Apports du projet . . . . .                                       | 37        |
| 5.7      | Difficultés rencontrées durant la phase d'investigation . . . . . | 38        |
| 5.8      | Conclusion . . . . .                                              | 38        |
| <b>6</b> | <b>Réalisation et implémentation</b>                              | <b>39</b> |
| 6.1      | Mise en place de l'environnement . . . . .                        | 39        |
| 6.1.1    | Poste de développement . . . . .                                  | 39        |
| 6.1.2    | Environnement tablette (Termux) . . . . .                         | 39        |
| 6.2      | Découverte et analyse du robot . . . . .                          | 39        |
| 6.3      | Reverse Engineering du système . . . . .                          | 40        |
| 6.3.1    | Analyse réseau . . . . .                                          | 40        |
| 6.3.2    | Identification des protocoles . . . . .                           | 40        |
| 6.3.3    | Étude des commandes . . . . .                                     | 41        |
| 6.4      | Développement du backend CYBEL . . . . .                          | 41        |
| 6.5      | Développement de l'application Android . . . . .                  | 42        |
| 6.5.1    | CybelTTSBridge . . . . .                                          | 42        |
| 6.5.2    | CybelVisitorKiosk . . . . .                                       | 42        |
| 6.5.3    | CybelVisitorKiosk . . . . .                                       | 42        |
| 6.6      | Développement de l'interface Web . . . . .                        | 43        |
| 6.6.1    | Interface opérateur ( <b>frontend/</b> ) . . . . .                | 43        |
| 6.6.2    | Interface visiteur ( <b>frontend-kiosk/</b> ) . . . . .           | 43        |
| 6.7      | Mise en place du système TTS . . . . .                            | 43        |
| 6.8      | Intégration des différents modules . . . . .                      | 44        |
| 6.9      | Conclusion . . . . .                                              | 44        |
| <b>7</b> | <b>Validation, résultats et analyse critique</b>                  | <b>45</b> |
| 7.1      | Stratégie de validation . . . . .                                 | 45        |
| 7.2      | Scénarios de tests . . . . .                                      | 45        |
| 7.3      | Résultats obtenus . . . . .                                       | 46        |
| 7.3.1    | Fonctionnalités livrées . . . . .                                 | 46        |
| 7.3.2    | Incidents terrain documentés . . . . .                            | 47        |
| 7.4      | Indicateurs de performance . . . . .                              | 47        |
| 7.5      | Contributions personnelles . . . . .                              | 48        |
| 7.6      | Difficultés rencontrées et solutions apportées . . . . .          | 48        |
| 7.7      | Limites du projet . . . . .                                       | 48        |
| 7.8      | Perspectives d'amélioration . . . . .                             | 49        |
| 7.9      | Conclusion . . . . .                                              | 49        |
|          | <b>CONCLUSION GÉNÉRALE</b>                                        | <b>50</b> |
|          | <b>BIBLIOGRAPHIE</b>                                              | <b>51</b> |

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>WEBOGRAPHIE</b>                                          | <b>52</b> |
| <b>ANNEXES</b>                                              | <b>53</b> |
| Annexe A - Captures d'écran . . . . .                       | 53        |
| Annexe B - Dépôt source du projet . . . . .                 | 56        |
| Annexe C - Commandes utiles . . . . .                       | 56        |
| Annexe D - Extraits de code significatifs . . . . .         | 57        |
| Annexe E - Diagrammes détaillés . . . . .                   | 58        |
| Annexe F - Documentation technique complémentaire . . . . . | 58        |



## Table des figures

|    |                                                                                        |    |
|----|----------------------------------------------------------------------------------------|----|
| 1  | Organigramme de l'HESTIM . . . . .                                                     | 3  |
| 2  | Robot CIOT TY1251D-03195 "CYBEL" . . . . .                                             | 6  |
| 3  | Architecture système globale du robot CIOT TY1251D-03195 . . . . .                     | 8  |
| 4  | Architecture générale du système CYBEL . . . . .                                       | 23 |
| 5  | Architecture en couches du système CYBEL . . . . .                                     | 24 |
| 6  | Diagramme de composants du système CYBEL . . . . .                                     | 25 |
| 7  | Diagramme de cas d'utilisation du système CYBEL . . . . .                              | 25 |
| 8  | Diagramme de séquence de la navigation vers un point sélectionné . . . . .             | 26 |
| 9  | Démarche de découverte . . . . .                                                       | 29 |
| 10 | Diagramme de Gantt du projet CYBEL . . . . .                                           | 30 |
| 11 | Logo de ROS . . . . .                                                                  | 32 |
| 12 | Logo de FastAPI . . . . .                                                              | 32 |
| 13 | Logo de Python . . . . .                                                               | 32 |
| 14 | Logo de ADB . . . . .                                                                  | 34 |
| 15 | Utilisation de rosbridge . . . . .                                                     | 34 |
| 16 | Identification de des services réseau avec Zenmmmap . . . . .                          | 35 |
| 17 | Logo de nmap . . . . .                                                                 | 36 |
| 18 | Logo de JADX . . . . .                                                                 | 36 |
| 19 | Interface Termux . . . . .                                                             | 37 |
| 20 | Structure du robot . . . . .                                                           | 40 |
| 21 | architecture Backend . . . . .                                                         | 42 |
| 22 | architecture Android . . . . .                                                         | 43 |
| 23 | Etape de la visite guidée . . . . .                                                    | 44 |
| 24 | Ancienne carte SLAM du laboratoire de l'interface opérateur . . . . .                  | 53 |
| 25 | Terminal vérification de connectivité et génération de clé SSH . . . . .               | 53 |
| 26 | Dashboard opérateur - mode robot réel ( <i>à insérer</i> ) . . . . .                   | 54 |
| 27 | Panneau visite guidée - page Visite ( <i>à insérer</i> ) . . . . .                     | 54 |
| 28 | Kiosque visiteur - écran d'accueil FR ( <i>à insérer</i> ) . . . . .                   | 55 |
| 29 | Kiosque visiteur - visite en cours ( <i>à insérer</i> ) . . . . .                      | 55 |
| 30 | Application <code>CybelVisitorKiosk</code> sur tablette ( <i>à insérer</i> ) . . . . . | 56 |
| 31 | Topologie réseau du robot ( <i>à insérer</i> ) . . . . .                               | 58 |

## Liste des tableaux

|    |                                                                              |    |
|----|------------------------------------------------------------------------------|----|
| 1  | Périmètre du projet : éléments inclus et exclus . . . . .                    | 11 |
| 2  | Acteurs du système et leurs rôles . . . . .                                  | 12 |
| 3  | Besoins fonctionnels du système . . . . .                                    | 13 |
| 4  | Besoins non fonctionnels du système . . . . .                                | 13 |
| 5  | Critères de réception et de validation . . . . .                             | 15 |
| 6  | Panorama des solutions existantes comparables . . . . .                      | 17 |
| 7  | Benchmark comparatif des solutions . . . . .                                 | 17 |
| 8  | Besoins fonctionnels du système CYBEL, synthèse par fonctionnalité . . . . . | 21 |
| 9  | Besoins non fonctionnels du système CYBEL, synthèse par critère . . . . .    | 22 |
| 10 | Organisation hebdomadaire indicative du projet . . . . .                     | 29 |
| 11 | Jalons principaux atteints (fin juin 2026) . . . . .                         | 31 |
| 12 | Éléments ROS principaux utilisés par CYBEL . . . . .                         | 31 |
| 13 | Logo de TypeScript et Vite . . . . .                                         | 33 |
| 14 | Logo de Android et Java . . . . .                                            | 33 |
| 15 | Canaux de communication du projet . . . . .                                  | 34 |
| 16 | Services réseau découverts sur le robot . . . . .                            | 35 |
| 17 | Synthèse des difficultés et solutions (phase d'investigation) . . . . .      | 38 |
| 18 | Commandes validées sur le robot CIOT . . . . .                               | 41 |
| 19 | Niveaux de validation du projet CYBEL . . . . .                              | 45 |
| 20 | Scénarios de tests et résultats . . . . .                                    | 46 |
| 21 | Incidents terrain et résolutions . . . . .                                   | 47 |
| 22 | Indicateurs de performance observés . . . . .                                | 47 |
| 23 | Perspectives d'amélioration priorisées . . . . .                             | 49 |
| 24 | Index de la documentation technique CYBEL . . . . .                          | 58 |

## LISTE DES ABRÉVIATIONS

|                  |                                                                                  |
|------------------|----------------------------------------------------------------------------------|
| <b>ADB</b>       | Android Debug Bridge                                                             |
| <b>API</b>       | Application Programming Interface (Interface de programmation applicative)       |
| <b>APK</b>       | Android Package (format d'installation des applications Android)                 |
| <b>DHCP</b>      | Dynamic Host Configuration Protocol                                              |
| <b>HRI</b>       | Human-Robot Interaction (Interaction Homme-Robot)                                |
| <b>HTTP</b>      | HyperText Transfer Protocol                                                      |
| <b>IHM</b>       | Interface Homme-Machine                                                          |
| <b>IP</b>        | Internet Protocol                                                                |
| <b>JSON</b>      | JavaScript Object Notation                                                       |
| <b>LiDAR</b>     | Light Detection and Ranging                                                      |
| <b>MQTT</b>      | Message Queuing Telemetry Transport                                              |
| <b>REST</b>      | Representational State Transfer                                                  |
| <b>RK3399</b>    | Référence du système sur puce (SoC) équipant la tête Android du robot            |
| <b>ROS</b>       | Robot Operating System                                                           |
| <b>rosbridge</b> | Composant ROS exposant les topics et services ROS via Web-Socket                 |
| <b>rosapi</b>    | Service ROS d'introspection des topics, services et types de messages            |
| <b>SDK</b>       | Software Development Kit (Kit de développement logiciel)                         |
| <b>SLAM</b>      | Simultaneous Localization and Mapping (Cartographie et localisation simultanées) |
| <b>SoC</b>       | System on Chip (Système sur puce)                                                |
| <b>TCP/IP</b>    | Transmission Control Protocol / Internet Protocol                                |
| <b>TTS</b>       | Text-To-Speech (Synthèse vocale)                                                 |
| <b>UML</b>       | Unified Modeling Language                                                        |
| <b>WebSocket</b> | Protocole de communication bidirectionnelle en temps réel sur TCP                |

## INTRODUCTION GÉNÉRALE

Les robots de service autonomes occupent une place croissante dans les environnements professionnels : accueil, guidage, téléprésence grâce à la maturité des technologies de navigation autonome et d'interaction homme-robot. Ces robots sont cependant généralement livrés avec un écosystème logiciel fermé : application propriétaire, protocole non documenté, absence d'API publique. Cette fermeture limite fortement la capacité des utilisateurs à adapter leur comportement à des besoins spécifiques.

C'est dans ce contexte que s'inscrit le robot de réception mobile **CIOT TY1251D-03195**, mis à disposition par HESTIM Engineering & Business School dans le cadre d'un stage. Composé d'un châssis de navigation sous ROS et d'une tête Android assurant l'interface visiteur, ce robot dépend entièrement d'une application propriétaire pour laquelle aucune documentation n'est disponible. Le projet **CYBEL**, objet de ce rapport, vise à concevoir une plateforme logicielle indépendante permettant de superviser, piloter et faire interagir ce robot sans recourir à l'application d'origine.

La problématique du stage est donc double : comment établir une communication fiable avec un système robotique fermé dont ni le protocole, ni les points d'accès réseau ne sont connus, et comment construire à partir de cette compréhension reconstruite une solution de contrôle robuste et extensible ? Le stage, d'une durée de trois à quatre mois, poursuit ainsi deux objectifs : procéder à une rétro-ingénierie méthodique du protocole de communication interne du robot, puis développer sur cette base une plateforme de commande complète, incluant une interface opérateur, une interface visiteur et des fonctionnalités d'interaction tactile et vocale. La démarche adoptée repose sur une approche itérative et prudente, où chaque hypothèse sur le fonctionnement du robot est systématiquement vérifiée avant d'être considérée comme acquise.

Le présent rapport restitue cette démarche en sept chapitres consacrés à la plateforme CYBEL : présentation de l'entreprise d'accueil, analyse du besoin et étude préalable, état de l'art, analyse et conception, méthodologie et planification, réalisation et implémentation, puis validation et analyse critique, avant une conclusion générale. Le module chatbot développé en parallèle par le second stagiaire n'est pas détaillé dans ce document.

## 1 Présentation de l'entreprise

L'HESTIM Engineering School (Hautes Études des Sciences et Techniques de l'ingénierie et du Management) est un établissement privé d'enseignement supérieur basé à Casablanca, au Maroc. Fondée en 2006, elle forme chaque année plusieurs centaines d'étudiants dans les domaines de l'ingénierie et du management. HESTIM fonctionne comme une entreprise de formation et de services académiques, structurée pour répondre aux besoins croissants des secteurs industriels et technologiques au Maroc et à l'international.

### 1.1 Activité principale et services

HESTIM propose des formations d'ingénieurs et de managers dans plusieurs spécialités, notamment :

- Génie Informatique et Intelligence Artificielle
- Génie Industriel et Logistique
- Génie Civil et Travaux Publics
- Management et Administration des Affaires

L'établissement offre des programmes d'enseignement initial, de formation continue et de doubles diplômes, en partenariat avec des universités et écoles étrangères. Il met également à disposition des laboratoires techniques et des équipements modernes : robots collaboratifs, capteurs 3D, robots de service Android ainsi que des services de recherche appliquée pour accompagner l'innovation.

### 1.2 Organisation interne

HESTIM est organisée autour de plusieurs pôles clés :

- **Pôle pédagogique**, qui regroupe les départements de formation spécialisés.
- **Pôle administratif**, qui assure la gestion des étudiants et des relations avec les entreprises partenaires.
- **Pôle recherche et développement**, qui coordonne les projets innovants menés en collaboration avec des entreprises ou dans le cadre de projets étudiants, dont le projet CYBEL fait partie.

#### 1.2.1 Organigramme de l'HESTIM

Pour illustrer la structure interne de l'HESTIM et mieux situer notre projet dans son environnement, voici un organigramme simplifié de l'établissement.

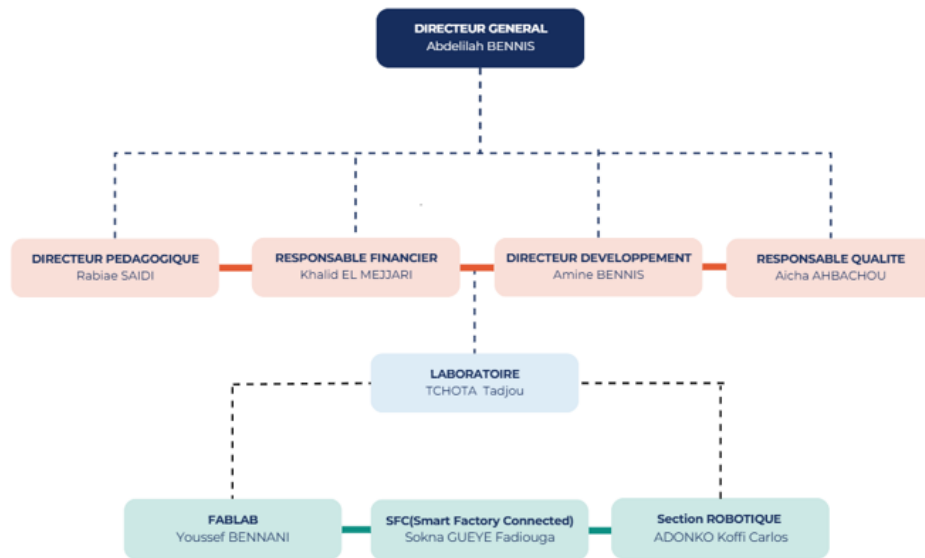


FIGURE 1 – Organigramme de l'HESTIM

### 1.3 Positionnement dans le secteur

HESTIM s'est imposée comme l'une des écoles d'ingénierie et de management les plus reconnues au Maroc. Elle bénéficie d'une réputation solide grâce à :

- la qualité de ses formations orientées vers les besoins concrets du marché ;
- ses nombreux partenariats avec des acteurs industriels nationaux et internationaux ;
- son engagement dans l'innovation pédagogique et technologique, notamment dans les domaines de la robotique et de l'intelligence artificielle.

HESTIM accueille un public varié, marocain et étranger, et cible des secteurs clés comme l'industrie, les technologies de l'information, le BTP et la logistique.

### 1.4 Valeurs et engagements

HESTIM place l'innovation, la qualité pédagogique et l'ouverture internationale au cœur de sa stratégie. L'école vise à former des ingénieurs et managers capables de répondre aux défis actuels et futurs de l'industrie, avec un fort accent sur l'éthique, l'esprit d'équipe et l'adaptabilité aux évolutions technologiques.

## 1.5 Lien avec notre stage

Le stage s'est déroulé au sein de HESTIM, considérée comme l'établissement d'accueil, car l'école offre un cadre idéal pour mettre en pratique les connaissances en génie informatique et en intelligence artificielle. La présence sur site du robot de service Android *CIOT TY1251D-03195* a permis d'expérimenter directement sur le matériel cible.

Le présent rapport documente la plateforme CYBEL, développée par Claudia Lusamote Kimfuta ; Ghassane Khalifi a mené en parallèle un travail complémentaire sur un module chatbot (branche dédiée du dépôt GitHub), non détaillé dans les chapitres techniques suivants. Grâce aux équipements du FabLab et à l'environnement de travail du laboratoire, ce stage a permis de mobiliser des compétences en rétro-ingénierie de protocoles, en développement d'API et en interfaces homme-robot, en adéquation avec le programme de la filière Génie Informatique.

## 1.6 Conclusion

La présentation de l'HESTIM Engineering School met en évidence un environnement académique et technologique favorable à la réalisation de projets innovants. Sa structure, ses équipements et en particulier la disponibilité du robot CYBEL et ses formations orientées vers l'industrie 4.0 ont permis de mener ce stage dans des conditions optimales, alliant rigueur académique et mise en situation professionnelle réelle.

## 2 Analyse du besoin et étude préalable

Ce chapitre présente le contexte général du projet CYBEL ainsi que l'analyse du besoin ayant conduit à la définition de la solution proposée. Il met en évidence l'architecture du système étudié, la problématique associée, les objectifs du projet ainsi que le cahier des charges fonctionnel et technique.

### 2.1 Contexte du projet

#### Présentation du robot CIOT TY1251D-03195

Le robot étudié est un **robot de réception mobile** commercialisé sous la référence **CIOT TY1251D-03195** illustré sur la figure 2. Il est composé de deux sous-systèmes matériels distincts, reliés en interne :

- un **châssis de navigation** fonctionnant sous **Linux embarqué avec ROS**, responsable de la localisation (SLAM), de la planification de trajectoire (`move_base`) et de l'exécution des commandes de mouvement ;
- un « **upper body** » **Android 7.1** (SoC RK3399, 2 Go de RAM, 16 Go de stockage), équipé d'un écran tactile 15,6" (1920×1080), de caméras (reconnaissance faciale, surveillance) et de haut-parleurs, qui héberge l'application propriétaire d'accueil et l'interface utilisateur standard du robot.

Le robot dispose d'un point d'accès WiFi propre (TY1251D-03195) permettant à un poste externe de rejoindre son réseau local. La topologie réseau s'est révélée plus complexe que ne le laissait supposer une simple architecture à deux hôtes : le châssis est joignable à l'adresse fixe 10.42.0.1, tandis que la tête Android obtient une adresse variable par DHCP sur un second segment 172.16.0.0/16 une première IP documentée (172.16.0.88) s'est révélée périmée en cours de projet. Les deux sous-systèmes sont par ailleurs reliés en interne par une liaison filaire `eth0` sur le segment 192.168.20.0/24 (192.168.20.1 pour la tête, 192.168.20.22 pour le châssis), indépendante du réseau WiFi exposé à l'extérieur.





FIGURE 2 – Robot CIOT TY1251D-03195 "CYBEL"

### Architecture générale du robot

Vue dans son ensemble, l'architecture du système peut se lire en quatre couches superposées, illustrées en figure 3 : une couche de présentation et d'interaction, une couche applicative et API, une couche de communication et de services réseau, et une couche matérielle.

**Couche de présentation et d'interaction (couche supérieure) :** Au sommet de l'architecture se trouve la couche de présentation, qui regroupe les interfaces utilisateur du robot. On y distingue trois composants : **l'application propriétaire Android**, boîte noire hors périmètre ; **l'interface opérateur web (CYBEL)**, développée dans le projet pour le contrôle et la supervision ; et **l'interface visiteur (mode kiosque)**, qui offre une interaction simplifiée en contexte public. Cette couche sert de point d'entrée homme-machine et s'appuie sur des échanges réseau vers les niveaux inférieurs, notamment HTTP et WebSocket. Côté Android, un module de **synthèse vocale (TTS natif)** et un mode **WebView kiosque** complètent les interfaces embarquées.

**Couche applicative et API (couche intermédiaire) :** La deuxième couche correspond à la logique applicative centrale, incarnée par l'**API CYBEL en FastAPI**. Elle orchestre les échanges entre interfaces et robot en exposant des endpoints REST et des flux WebSocket. Cette API utilise soit

un **SDK Python réel** pour la communication directe avec le robot, soit un **SDK Python simulé (*mock*)** pour le développement et les tests, tous deux basés sur une interface abstraite commune (**RobotBackend**).

**Couche de communication et de services réseau :** La troisième couche regroupe les interfaces réseau du robot et les services qui exposent ses fonctionnalités internes, notamment **rosbridge** (WebSocket, port 9090), **rosapi**, **MQTT** (port 1883) et **ADB** (Android Debug Bridge). Elle fait le lien entre la logique applicative CYBEL et le système robotique réel.

**Couche matérielle et système robotique :** À la base de l'architecture se trouve la couche matérielle, composée des deux sous-systèmes principaux du robot. D'une part, le **châssis de navigation** sous Linux embarqué avec ROS assure les fonctions de mobilité : il intègre les modules de SLAM, de planification de trajectoire (**move\_base**) et les capteurs de perception tels que le LiDAR, et reste accessible via l'adresse réseau interne **10.42.0.1**. D'autre part, le **module supérieur Android** constitue la partie interactive du robot : il regroupe l'écran tactile, les caméras, les haut-parleurs et les applications utilisateur, et reste accessible via un réseau interne distinct (**172.16.0.0/16**).

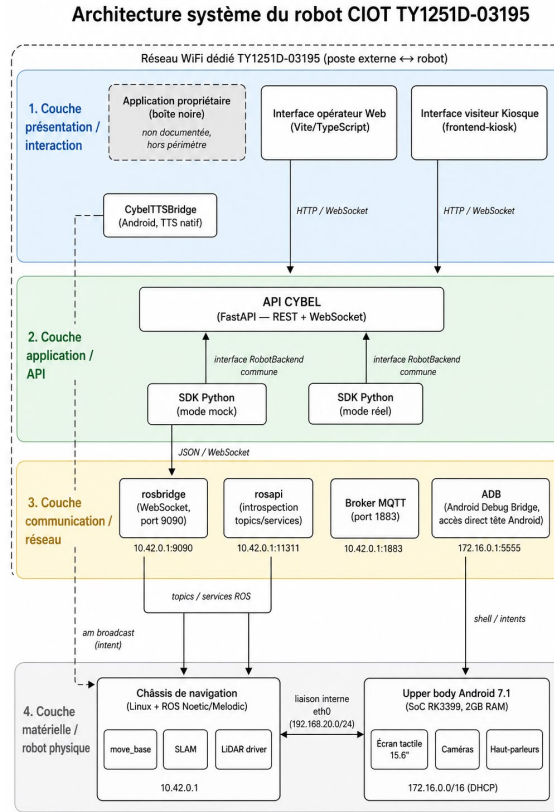


FIGURE 3 – Architecture système globale du robot CIOT TY1251D-03195

**Positionnement du système CYBEL dans l'architecture.** CYBEL se place dans les couches supérieures, en remplaçant l'application propriétaire et en ajoutant des fonctions. Il utilise l'API FastAPI et le SDK Python pour la logique de commande, rosbridge et MQTT pour la communication, et s'appuie sur le matériel robotique pour exécuter les actions. Cette solution intermédiaire sépare l'interface utilisateur du système robotique tout en conservant la compatibilité avec l'infrastructure existante.

### Cadre et déroulement du stage

L'établissement disposant de ce robot souhaite l'utiliser pour des scénarios d'accueil et de démonstration personnalisés : annonces vocales spécifiques, déplacements vers des points définis par l'utilisateur, supervision à distance. Le projet est encadré par HESTIM Engineering & Business School et s'adresse à des étudiants du cycle ingénieur (dont la 1<sup>re</sup> année). Il est prévu sur une durée de trois à quatre mois, structuré en quatre phases : (1) compréhension

de l'architecture et établissement de la connectivité réseau, (2) exploration et analyse du protocole de communication interne, (3) développement de l'interface de commande et des modules d'interaction, et (4) intégration, tests et validation finale.

### Limites de la solution existante

L'application propriétaire installée sur l'*upper body* ne permet pas le type de personnalisation recherché : elle constitue une boîte noire, sans documentation, sans API exposée et sans code source disponible. Le constructeur ne fournit par ailleurs aucune documentation technique sur les protocoles de communication internes, ce qui interdit toute extension ou adaptation directe de la solution d'origine.

## 2.2 Problématique

La problématique centrale du projet peut être formulée ainsi :

*Comment concevoir une plateforme de commande et d'interaction tierce, fonctionnellement équivalente ou supérieure à l'application propriétaire d'un robot de service Android, en l'absence de toute documentation officielle sur ses interfaces de communication internes ?*

Cette problématique se décline en plusieurs sous-questions :

- Quels sont les **points d'accès réseau** (adresses IP, ports, protocoles) exposés par le robot, et lesquels sont effectivement exploitables sans authentification propriétaire ?
- Quel est le **format des messages** échangés entre l'interface Android et le système de navigation (ROS), et peut-il être reconstruit par observation et introspection plutôt que par décompilation de l'application ?
- Comment **distinguer une commande effectivement traitée par le robot d'une commande silencieusement ignorée**, dans un système où le protocole de transport (*rosbridge*) ne renvoie pas systématiquement d'erreur explicite ?
- Comment concevoir une architecture logicielle qui reste **utilisable et testable sans accès permanent au robot physique** (contraintes de disponibilité du matériel), tout en restant fidèle au comportement réel une fois connectée ?
- Dans quelle mesure les fonctionnalités d'**interaction homme-robot** (synthèse vocale, reconnaissance vocale, retour visuel) peuvent-elles

être reconstruites lorsque le canal correspondant n'est pas exposé par le système de navigation (ROS) et nécessite un accès direct au sous-système Android ?

## 2.3 Objectifs du projet

### 2.3.1 Objectif général

Concevoir et développer une **plateforme web indépendante** de commande, de supervision et d'interaction pour le robot CIOT TY1251D-03195, sans dépendre de l'application Android propriétaire, en documentant de façon rigoureuse le **protocole de communication** reconstruit (endpoints, topics, services, formats de messages) afin que ce travail puisse être réutilisé ou étendu.

### 2.3.2 Objectifs spécifiques

1. **Établir la connectivité** avec le robot (réseau WiFi dédié, identification des hôtes et ports actifs).
2. **Identifier les points de communication** exposés : WebSocket `rosbridge` (port 9090), broker MQTT (port 1883), services FTP/SSH, interfaces web internes (:8082, :8088).
3. **Capturer et analyser le trafic réseau** afin de reconstituer les topics et services ROS pertinents (mouvement, navigation, télémétrie, capteurs).
4. **Décoder et reconstruire les commandes de contrôle** du mouvement et de la navigation (vitesse, position cible, annulation de trajectoire).
5. **Développer une interface de commande web** (backend FastAPI + frontend TypeScript) permettant la supervision temps réel et l'envoi de commandes.
6. **Implémenter des fonctionnalités d'interaction homme-robot de base** : navigation par sélection de points ou clic sur carte, retour vocal (TTS) et commande vocale opérateur.
7. **Mettre en place un mode de simulation (« mock »)** permettant de développer et tester l'interface sans dépendre de la disponibilité physique du robot.
8. **Intégrer l'ensemble des composants** (SDK Python, API, interface web) dans un système cohérent et démontrable.

## 2.4 Cahier des charges

### 2.4.1 Périmètre du projet

Le tableau 1 délimite les activités prises en charge par le projet de celles volontairement laissées hors périmètre.

| Inclus dans le périmètre                                                                                               | Exclu du périmètre                                                              |
|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Identification et documentation des canaux de communication du robot ( <code>rosbridge</code> , MQTT, services réseau) | Modification du firmware ou du logiciel embarqué du robot                       |
| Développement d'un SDK Python d'accès au robot (mode simulé et mode réel)                                              | Développement d'une application Android de remplacement (optionnel, non engagé) |
| Développement d'une API de commande/supervision (FastAPI)                                                              | Décompilation ou rétro-ingénierie de l'application Android propriétaire         |
| Développement d'une interface web opérateur (Vite/TypeScript)                                                          | Déploiement en production / hébergement distant                                 |
| Fonctionnalités de navigation (point nommé, clic sur carte, téléopération)                                             | Comportements autonomes avancés (planification de mission, multi-robot)         |
| Interaction de base : synthèse vocale (TTS) et commande vocale opérateur                                               | Vision par caméra, reconnaissance faciale, chatbot (extensions optionnelles)    |
| Documentation technique et rapport de stage                                                                            | Formation des utilisateurs finaux / exploitation au long cours                  |

TABLE 1 – Périmètre du projet : éléments inclus et exclus

### 2.4.2 Acteurs du système

Le tableau 2 identifie les principaux acteurs impliqués dans le projet ainsi que leur rôle respectif.

| <b>Acteur</b>                                                  | <b>Rôle</b>                                                                                                                      |
|----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Étudiant(e) stagiaire - plateforme CYBEL (C. LUSAMOTE KIMFUTA) | Rétro-ingénierie, SDK, backend, interfaces web, Android, déploiement Termux, documentation technique, contenu du présent rapport |
| Étudiant(e) stagiaire - chatbot (G. KHALIFI)                   | Développement d'un module chatbot sur la branche <code>chatbot-cybel-projet-2026</code> du dépôt GitHub (hors périmètre CYBEL)   |
| Encadrante pédagogique (Mme MESKINI, HESTIM)                   | Suivi pédagogique, validation académique du rapport                                                                              |
| Encadrant en entreprise (M. Tula, HESTIM)                      | Suivi technique du robot, validation des objectifs sur le terrain                                                                |
| Établissement d'accueil                                        | Mise à disposition du robot et de l'environnement de travail                                                                     |
| Opérateur final                                                | Supervision et pilotage du robot via la plateforme CYBEL                                                                         |
| Robot CIOT TY1251D-03195                                       | Système cible - serveur de télémétrie et de commandes via <code>rosbridge</code> /MQTT                                           |

TABLE 2 – Acteurs du système et leurs rôles

### 2.4.3 Besoins fonctionnels

Le tableau 3 liste les exigences fonctionnelles du système, accompagnées de leur niveau de priorité.

| Réf.  | Exigence                                                                                       | Priorité    |
|-------|------------------------------------------------------------------------------------------------|-------------|
| EF-01 | Superviser en temps réel l'état du robot (batterie, position, statut, mode de navigation)      | Essentielle |
| EF-02 | Afficher la carte SLAM, le LiDAR et les personnes détectées                                    | Essentielle |
| EF-03 | Naviguer vers un point nommé prédéfini                                                         | Essentielle |
| EF-04 | Naviguer vers une coordonnée choisie par clic sur la carte, avec prise en compte des obstacles | Essentielle |
| EF-05 | Téléopérer manuellement le robot (vitesse linéaire/angular) avec arrêt d'urgence               | Essentielle |
| EF-06 | Déclencher une annonce vocale sur le robot (TTS)                                               | Souhaitable |
| EF-07 | Donner des commandes vocales à l'opérateur via le micro du navigateur                          | Souhaitable |
| EF-08 | Configurer les paramètres de fonctionnement (vitesse, mode de déplacement)                     | Souhaitable |
| EF-09 | Fonctionner en mode simulation complet sans robot connecté                                     | Essentielle |

TABLE 3 – Besoins fonctionnels du système

#### 2.4.4 Besoins non fonctionnels

Le tableau 4 présente les exigences non fonctionnelles, relatives notamment à la performance, à la robustesse et à la séparation des responsabilités au sein de l'architecture.

| Réf.   | Exigence                                                                            | Priorité    |
|--------|-------------------------------------------------------------------------------------|-------------|
| ENF-01 | Latence d'affichage de la télémétrie de l'ordre de quelques centaines de ms maximum | Essentielle |
| ENF-02 | Reconnexion automatique au robot après perte de connexion réseau                    | Essentielle |
| ENF-03 | Séparation stricte entre couche d'accès robot (SDK), API et présentation            | Essentielle |
| ENF-04 | Toute commande de mouvement/navigation doit être annulable à tout moment            | Essentielle |
| ENF-05 | Aucune action exploratoire ne doit perturber le fonctionnement normal du robot      | Essentielle |

TABLE 4 – Besoins non fonctionnels du système



### 2.4.5 Contraintes du projet

- **Contraintes techniques** : absence de documentation officielle du protocole ; dépendance à un réseau WiFi dédié et potentiellement instable (latence variable, de 7 à 1654 ms observés) ; version Android embarquée obsolète (7.1) limitant les outils de débogage disponibles.
- **Contraintes matérielles** : un seul exemplaire de robot disponible, partagé avec d'autres usages de l'établissement ; disponibilité limitée pour les tests.
- **Contraintes organisationnelles** : durée de stage limitée à trois à quatre mois ; deux stagiaires avec des périmètres distincts (plateforme CYBEL et chatbot) ; encadrement pédagogique et terrain périodique.
- **Contraintes de sécurité et d'éthique** : les canaux **rosbridge** et **MQTT** du robot étant accessibles sans authentification, toute manipulation doit respecter un principe de **précaution** (vérification en mode simulation avant toute commande réelle, mécanisme d'annulation disponible) ; les tentatives d'accès aux comptes système (SSH/FTP) doivent rester mesurées pour ne pas provoquer de blocage.
- **Contraintes de propriété intellectuelle** : aucune décompilation du logiciel propriétaire n'est entreprise ; seule l'observation du trafic réseau et l'introspection des interfaces standard (ROS, MQTT) sont utilisées.

### 2.4.6 Livrables attendus

Conformément au sujet de stage, les livrables attendus en fin de projet sont :

1. Une **interface de communication fonctionnelle** avec le robot (SDK + client **rosbridge**/MQTT).
2. Un **système de contrôle du mouvement** opérationnel (téléopération + navigation).
3. Une **interface utilisateur** web opérateur.
4. Un **module d'interaction de base** (tactile et/ou vocal).
5. Un **rapport technique** incluant l'architecture du système et l'analyse du protocole.

### 2.4.7 Critères de réception et de validation

Le tableau 5 précise, pour chaque critère d'acceptation, la modalité de vérification associée.

| Critère                                           | Modalité de vérification                                                                                            |
|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Connexion au robot établie de façon reproductible | Connexion <code>rosbridge</code> réussie depuis le réseau WiFi du robot, vérifiable via <code>/rosapi/topics</code> |
| Télémétrie affichée en temps réel                 | Observation du tableau de bord (position, batterie, statut) à jour                                                  |
| Navigation vers un point ou une coordonnée        | Envoi d'une commande de navigation suivie d'un changement d'état ( <code>/navi_status</code> ) cohérent             |
| Téléopération et arrêt d'urgence                  | Test en mode simulation puis, si possible, sur robot réel à vitesse réduite                                         |
| Fonctionnement en mode simulation                 | Lancement avec <code>ROBOT MOCK=true</code> , toutes les fonctionnalités principales accessibles                    |
| Interaction vocale                                | TTS robot <b>ou</b> solution de repli (voix opérateur via navigateur) fonctionnelle                                 |
| Documentation                                     | Présence et cohérence de <code>README.md</code> , <code>docs/INTERFACE.md</code> et du présent rapport              |

TABLE 5 – Critères de réception et de validation

## 2.5 Solution proposée

Face à ce besoin, la solution retenue consiste en une plateforme logicielle indépendante, structurée autour d'une couche d'accès au robot (SDK Python interchangeable mock/réel), d'une API de commande et de supervision, et d'interfaces utilisateur web dédiées à l'opérateur et au visiteur. L'analyse fonctionnelle détaillée, l'architecture logicielle complète et les diagrammes de conception associés sont présentés au Chapitre 4 (*Analyse et conception du système*).

## 2.6 Conclusion

Ce chapitre a permis de poser le cadre du projet CYBEL : l'architecture du robot CIOT TY1251D-03195 et les limites de sa solution propriétaire, la problématique de rétro-ingénierie et de conception qui en découle, les objectifs poursuivis, ainsi qu'un cahier des charges détaillant acteurs, besoins, contraintes, livrables et critères de validation. Le chapitre suivant situe ce travail par rapport à l'état de l'art en robotique de service, architectures logicielles robotiques et interaction homme-robot.

## 3 État de l'art

Ce chapitre situe le projet CYBEL par rapport aux travaux et solutions existants. Il présente les fondements théoriques de la robotique de service ainsi que les solutions logicielles et matérielles comparables, afin de mettre en évidence les limites des approches actuelles et de justifier les choix architecturaux retenus.

### 3.1 Fondements théoriques

La conception de la plateforme s'appuie sur plusieurs corpus de référence :

- **Architecture logicielle robotique** : le modèle ROS (topics, services, actions) proposé par Quigley et al. constitue le paradigme de communication analysé dans ce projet.
- **Navigation autonome** : les concepts de cartes d'occupation et de planification (costmaps, DWA) décrits dans *Probabilistic Robotics* (Thrun, Burgard & Fox, 2005) servent de base à l'interprétation du module `move_base`.
- **Protocole rosbridge** : le standard JSON de communication ROS (subscribe, publish, call\_service) est utilisé comme référence pour la reconstruction des échanges observés.
- **Interaction homme-robot (HRI)** : les principes de supervision en temps réel et de sécurité opérationnelle (priorité à l'arrêt d'urgence) guident la conception de l'interface CYBEL.
- **Sécurité IoT** : les recommandations de l'OWASP IoT Top 10 permettent d'analyser les risques liés aux canaux non authentifiés observés sur le système.

### 3.2 État des solutions existantes

Les solutions suivantes représentent les principales approches disponibles dans l'écosystème robotique et du robot étudié.

### 3.3 Analyse comparative

### 3.4 Limites des solutions existantes

Les solutions étudiées présentent plusieurs limites majeures :

- Absence d'extensibilité des systèmes propriétaires
- Outils ROS de trop bas niveau pour des usages métier
- Aucune solution ne propose de mode déconnecté du robot

| <b>Solution</b>                               | <b>Description</b>                                        | <b>Accessibilité</b>                 |
|-----------------------------------------------|-----------------------------------------------------------|--------------------------------------|
| Application propriétaire (upper body Android) | Interface tactile constructeur pour accueil et navigation | Fermée, sans API ni documentation    |
| Interface web embarquée (:8082)               | Outil de gestion des cartes SLAM                          | Accessible localement, non documenté |
| Interface de debug (:8088)                    | Interface interne de diagnostic                           | Non documentée, usage inconnu        |
| rosbridge / RViz / Foxglove                   | Outils ROS de visualisation et contrôle                   | Open-source mais génériques          |
| SDK robots commerciaux (Temi, Pepper, etc.)   | SDK propriétaires pour robots de service                  | Documentés mais verrouillés          |

TABLE 6 – Panorama des solutions existantes comparables

| <b>Critère</b>             | <b>App. propriétaire</b> | <b>RViz / Foxglove</b> | <b>CYBEL</b>               |
|----------------------------|--------------------------|------------------------|----------------------------|
| Documentation              | Aucune                   | Open-source            | Documenté (rapport + code) |
| Accès distant web          | Non                      | Partiel                | Oui                        |
| Personnalisation scénarios | Non                      | Non                    | Oui                        |
| Mode simulation            | Non                      | Partiel                | Oui (mock)                 |
| Navigation interactive     | Oui                      | Oui                    | Oui + logique métier       |
| Interaction vocale         | Oui                      | Non                    | Partiel / évolutif         |

TABLE 7 – Benchmark comparatif des solutions

— Documentation constructeur insuffisante nécessitant rétro-ingénierie

### 3.5 Conclusion

Ce chapitre a permis de situer le projet CYBEL par rapport aux fondements théoriques de la robotique de service architecture ROS, navigation autonome, protocole `rosbridge`, interaction homme-robot et sécurité des objets connectés ainsi qu’aux solutions existantes, qu’elles soient proprié-

taires (application Android constructeur) ou génériques (rosbridge, RViz, Foxglove, SDK de robots commerciaux). L'analyse comparative a montré qu'aucune de ces solutions ne répond pleinement aux besoins identifiés au Chapitre 2 : extensibilité, logique métier orientée accueil, supervision de haut niveau et mode de développement déconnecté du robot physique. Ces limites confirment la pertinence du projet CYBEL et orientent directement les choix de conception présentés au Chapitre 4, qui détaille l'analyse fonctionnelle et l'architecture logicielle de la solution retenue.

## 4 Analyse et conception du système

Ce chapitre présente l'analyse fonctionnelle du système CYBEL ainsi que les choix de conception retenus pour répondre au cahier des charges défini au chapitre 2, à la lumière de l'analyse critique des solutions existantes présentée au chapitre 3. Il détaille successivement les acteurs et les besoins identifiés, l'architecture générale et logicielle de la plateforme, les diagrammes de conception (composants, cas d'utilisation, séquence) et le modèle des flux de communication entre les différents sous-systèmes.

### 4.1 Analyse fonctionnelle

Après l'étude du besoin et l'analyse des solutions existantes, il est nécessaire de définir précisément le fonctionnement du système à concevoir. Cette phase d'analyse fonctionnelle permet d'identifier les acteurs intervenant dans le système, les services qui leur sont proposés ainsi que les interactions nécessaires à la réalisation des différentes fonctionnalités.

Le projet CYBEL vise à fournir une plateforme de supervision, de commande et d'interaction destinée au robot de service CIOT TY1251D-03195, déployé au sein du FabLab de HESTIM. Cette plateforme doit permettre aux opérateurs de piloter le robot, de suivre son état en temps réel et de gérer les interactions avec les visiteurs, tout en assurant une communication fiable avec les différents composants matériels et logiciels du système.

L'analyse fonctionnelle constitue ainsi une étape essentielle avant la conception détaillée de l'architecture, puisqu'elle permet de traduire les besoins exprimés dans le cahier des charges (section 3 et suivantes) en fonctionnalités concrètes, et de les rattacher explicitement aux exigences EF/ENF formulées au chapitre 2.

#### 4.1.1 Identification des acteurs

L'étude des besoins a permis d'identifier trois acteurs principaux intervenant dans le fonctionnement du système CYBEL.

**L'opérateur :** L'opérateur représente l'utilisateur chargé de l'administration et de la supervision du robot. Il dispose d'une interface lui permettant de consulter l'état du robot, de visualiser sa position sur la carte, d'accéder aux informations de télémétrie, de lancer des scénarios de navigation, de téléopérer le robot, de déclencher des annonces vocales et de gérer les paramètres du système. Son rôle est central puisque c'est lui qui assure le contrôle opérationnel du robot au sein du FabLab.

**Le visiteur :** Le visiteur interagit avec le robot à travers l'interface embarquée sur la tablette Android. Cette interface lui permet notamment de découvrir les différents espaces du FabLab, de sélectionner des points d'intérêt, de consulter des informations de présentation et d'interagir avec le robot lors des démonstrations. Le visiteur n'a pas accès aux fonctionnalités d'administration ou de contrôle du système.

**Le système CYBEL :** Le système CYBEL constitue un acteur autonome chargé de coordonner les différents composants logiciels. Il assure notamment la collecte des données du robot, la diffusion de la télémétrie, la gestion des communications, la reconnexion automatique en cas de perte de liaison, la synchronisation des données entre les interfaces, ainsi que le déclenchement des services associés à la navigation et à l'interaction.

#### 4.1.2 Besoins fonctionnels

L'analyse des besoins réalisée au cours des phases préliminaires du projet a permis d'identifier les principales fonctionnalités que le système CYBEL doit assurer afin de répondre aux attentes des utilisateurs et aux contraintes du robot. Les besoins fonctionnels retenus, repris du cahier des charges établi au chapitre 2 (réf. EF-01 à EF-09), sont présentés dans le tableau 8 avec un rajout de quelque fonctionnalité identifier par la suite (EF-10 à EF-11).

TABLE 8 – Besoins fonctionnels du système CYBEL, synthèse par fonctionnalité

| Réf.            | Fonctionnalité             | Description                                                                                                                                                                                      |
|-----------------|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| EF-01,<br>EF-02 | Supervision du robot       | Consultation en temps réel du niveau de batterie, de la position du robot, de son état de navigation, de la carte SLAM, des données LiDAR, des visiteurs détectés et des messages de diagnostic. |
| EF-03,<br>EF-04 | Navigation autonome        | Envoi du robot vers un point prédéfini, navigation vers une position sélectionnée sur la carte, suivi de mission et annulation d'une tâche en cours.                                             |
| EF-05           | Téléopération              | Contrôle manuel des déplacements du robot via une interface de commande, avec arrêt d'urgence.                                                                                                   |
| EF-06,<br>EF-07 | Interaction Homme-Robot    | Déclenchement d'annonces vocales (TTS), diffusion de messages d'accueil et commandes vocales données par l'opérateur.                                                                            |
| EF-08           | Configuration du système   | Modification des paramètres de fonctionnement tels que les vitesses de déplacement, les paramètres de navigation et les réglages de communication.                                               |
| EF-09           | Mode simulation            | Exécution du système sans robot physique afin de faciliter les phases de développement et de test.                                                                                               |
| EF-10           | Interface visiteur kiosque | Accès tactile dédié, indépendant de l'interface opérateur, proposant des actions d'accueil et une foire aux questions bilingue (FR/EN) destinées au visiteur.                                    |
| EF-11           | Déploiement autonome       | Exécution de la plateforme directement sur la tablette du robot, sans dépendance à un poste de développement externe.                                                                            |

#### 4.1.3 Besoins non fonctionnels

Au-delà des fonctionnalités attendues, le système doit respecter un ensemble d'exigences de qualité garantissant sa robustesse, sa fiabilité et son utilisabilité. Les besoins non fonctionnels identifiés, repris du chapitre 2 (réf. ENF-01 à ENF-05), sont présentés dans le tableau 9.



TABLE 9 – Besoins non fonctionnels du système CYBEL, synthèse par critère

| Réf.   | Critère                            | Description                                                                                                                                                                                 |
|--------|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ENF-01 | Performance                        | Mise à jour rapide des informations (télémétrie de l'ordre de quelques centaines de millisecondes maximum) afin d'assurer une supervision efficace et une interaction fluide avec le robot. |
| ENF-02 | Fiabilité                          | Maintien de la continuité des communications et reconnexion automatique en cas d'interruption réseau.                                                                                       |
| ENF-03 | Maintenabilité                     | Architecture modulaire, séparant strictement la couche d'accès au robot, l'API et la présentation, facilitant l'évolution et la maintenance du système.                                     |
| ENF-04 | Sécurité opérationnelle            | Possibilité d'annuler à tout moment toute commande de mouvement ou de navigation en cours.                                                                                                  |
| ENF-05 | Innocuité des phases exploratoires | Les scripts utilisés lors de la rétro-ingénierie du protocole ne doivent en aucun cas perturber le fonctionnement normal du robot.                                                          |
|        | Ergonomie                          | Interfaces intuitives et adaptées aussi bien aux opérateurs qu'aux visiteurs.                                                                                                               |

## 4.2 Architecture générale de CYBEL

Afin de répondre aux besoins identifiés, une architecture modulaire a été retenue. Celle-ci repose sur plusieurs composants indépendants communiquant entre eux à travers des interfaces clairement définies. L'objectif principal de cette architecture est de découpler les interfaces utilisateur du système robotique afin de faciliter les tests, la maintenance et l'évolution de la plateforme.

Comme illustré en figure 4, l'architecture globale du système s'articule, d'un point de vue physique et déploiement, autour de quatre ensembles principaux : le robot CIOT TY1251D-03195, le backend CYBEL, les interfaces utilisateur et les applications Android embarquées. La sous-section suivante propose une seconde vue de ce même système, cette fois d'un point de vue logique, organisée en couches logicielles.

## 4.3 Architecture logicielle

L'architecture logicielle retenue suit une approche en couches permettant de séparer les responsabilités fonctionnelles et techniques du système. Elle est organisée autour de trois couches principales, présentées en figure 5.

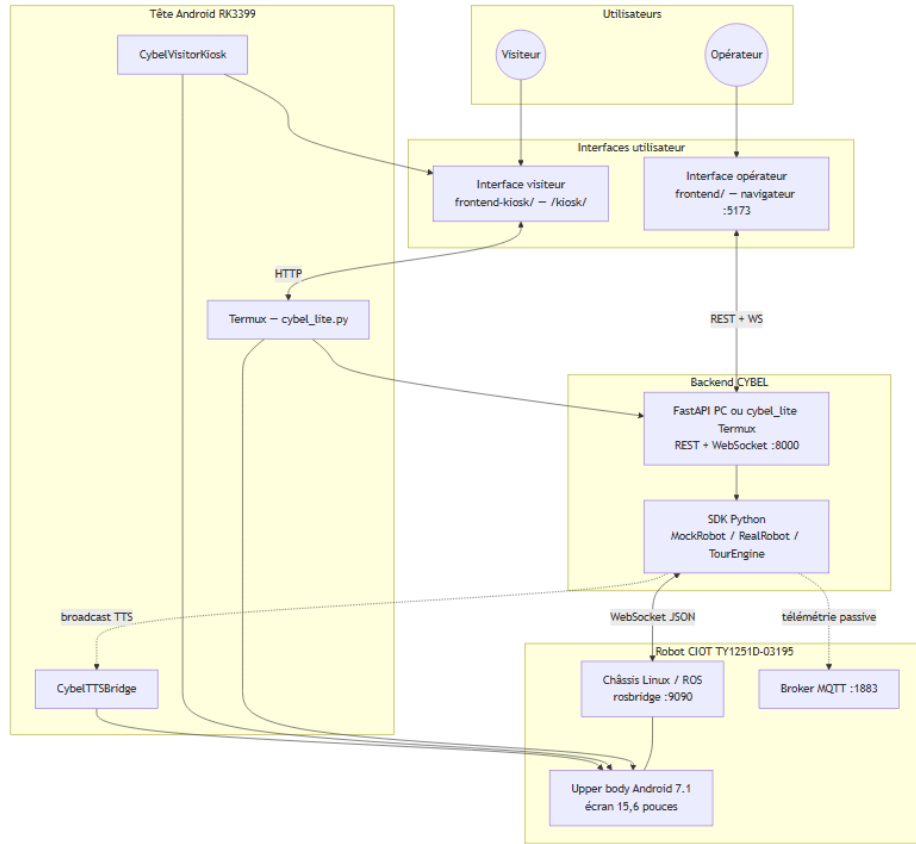


FIGURE 4 – Architecture générale du système CYBEL

#### 4.3.1 Couche d'accès au robot (SDK)

Cette couche constitue l'abstraction du robot. Elle est responsable de la communication avec *robridge*, de la gestion du protocole reconstruit, de l'accès aux données de télémétrie et de l'exécution des commandes. Deux implémentations sont proposées : une implémentation réelle et une implémentation simulée. Cette approche permet de développer et de tester la plateforme sans dépendre de la disponibilité permanente du robot.

#### 4.3.2 Couche API

Cette couche repose sur FastAPI et constitue le point d'entrée principal du système. Elle expose des services REST, des services WebSocket ainsi que les mécanismes de gestion des événements. Elle joue également le rôle d'intermédiaire entre les interfaces utilisateur et le SDK.

### 4.3.3 Couche présentation

Cette couche regroupe l'interface opérateur, l'interface visiteur et les applications Android complémentaires. Elle permet l'interaction entre les utilisateurs et le système.

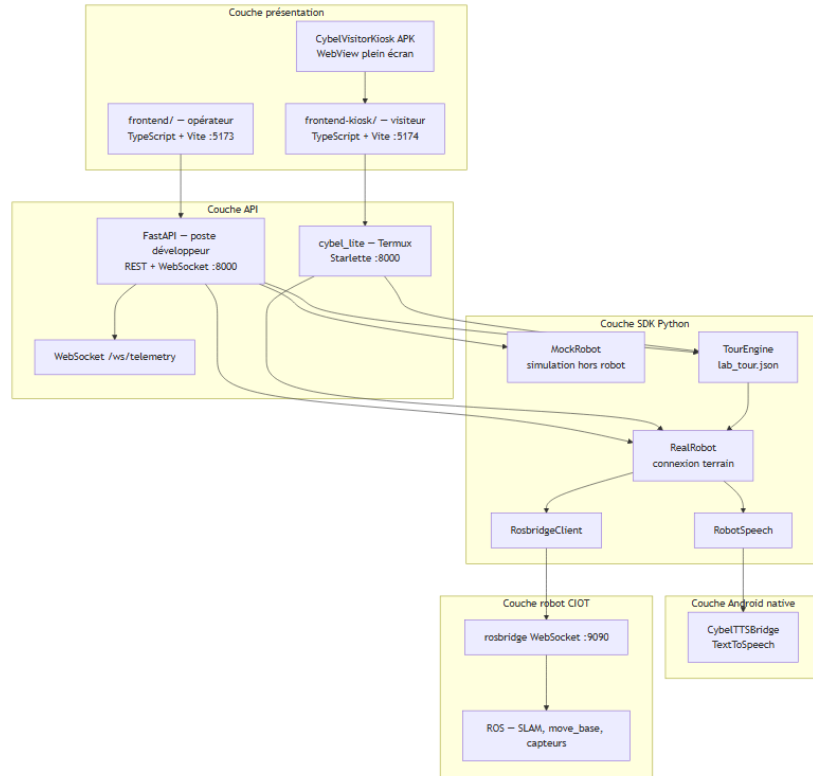


FIGURE 5 – Architecture en couches du système CYBEL

## 4.4 Diagramme des composants

Le diagramme des composants (figure 6) met en évidence l'organisation des différents modules logiciels du système ainsi que leurs dépendances. Il montre notamment les interfaces opérateur et visiteur, le backend FastAPI, le SDK robot, *rosbridge*, le système Android embarqué ainsi que les modules TTS. Cette représentation permet de visualiser clairement la répartition des responsabilités entre les différents composants.

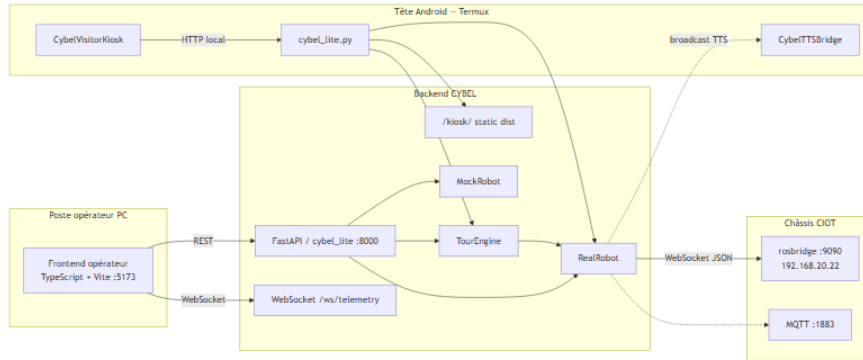


FIGURE 6 – Diagramme de composants du système CYBEL

## 4.5 Diagrammes de cas d'utilisation

Les diagrammes de cas d'utilisation (figure 7) permettent de représenter les interactions entre les acteurs identifiés et les fonctionnalités proposées par le système. Ils mettent en évidence les actions réalisées par l'opérateur, les interactions du visiteur, ainsi que les traitements automatiques assurés par CYBEL.



FIGURE 7 – Diagramme de cas d'utilisation du système CYBEL

## 4.6 Diagrammes de séquence

Les diagrammes de séquence (figure 8) permettent de décrire l'enchaînement des échanges entre les différents composants du système lors de l'exécution d'une fonctionnalité. Parmi les scénarios étudiés figurent la navigation vers un point sélectionné, l'envoi d'une commande de déplacement, le déclenchement d'une annonce vocale, ainsi que la diffusion de la télémétrie vers l'interface opérateur. Ces diagrammes facilitent la compréhension des mécanismes internes mis en œuvre lors des communications entre le robot, le backend et les interfaces utilisateur.

## 4.7 Modèle des flux de communication

### 4.7.1 Robot ↔ rosbridge

*rosbridge* constitue la passerelle principale entre la plateforme CYBEL et le système de navigation du robot. Les échanges reposent sur le protocole

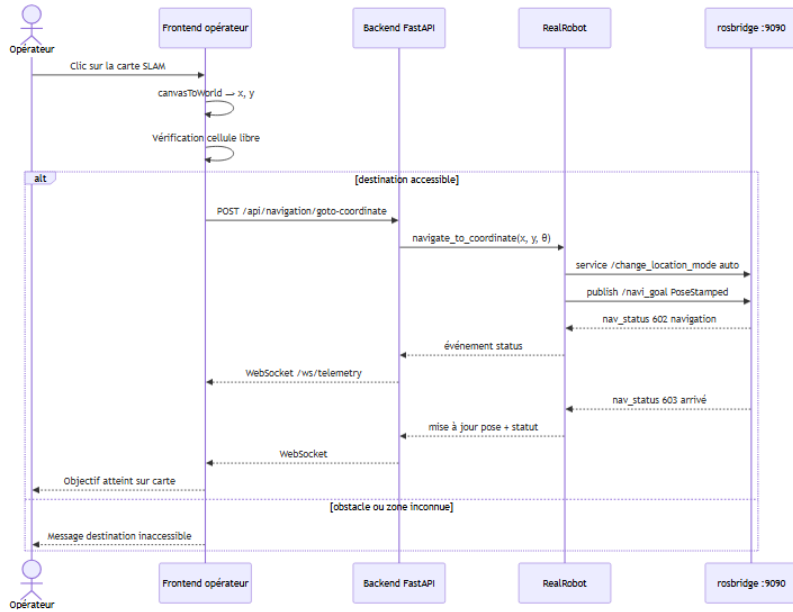


FIGURE 8 – Diagramme de séquence de la navigation vers un point sélectionné

WebSocket et permettent la réception des données de télémétrie, l'abonnement aux topics ROS, l'appel de services, ainsi que l'envoi de commandes de navigation et de mouvement.

#### 4.7.2 rosbridge ↔ Backend

Le backend agit comme intermédiaire entre *rosbridge* et les interfaces utilisateur. Il collecte les informations provenant du robot, les traite puis les redistribue vers les clients connectés. Il assure également la gestion des commandes envoyées par les utilisateurs.

#### 4.7.3 Backend ↔ Interfaces utilisateur

Les interfaces communiquent avec le backend à travers des requêtes REST pour les actions ponctuelles et des WebSockets pour les mises à jour temps réel. Cette architecture garantit une supervision réactive et une faible latence lors de l'affichage des données.

#### 4.7.4 Backend ↔ Applications Android

Les applications Android développées dans le cadre du projet permettent d'étendre les fonctionnalités du système, notamment pour la synthèse vocale,

l’affichage du mode kiosque et les interactions avec la tablette embarquée.

## **4.8 Conclusion**

Ce chapitre a permis de présenter l’analyse fonctionnelle et la conception générale du système CYBEL. Les besoins identifiés, repris et détaillés à partir du cahier des charges du chapitre 2, ont conduit à la définition d’une architecture modulaire organisée autour d’un SDK robot, d’une API centralisée et de plusieurs interfaces utilisateur. Les différents diagrammes présentés permettent de visualiser les interactions entre les acteurs et les composants du système ainsi que les flux de communication mis en œuvre. Cette phase de conception constitue la base de la démarche méthodologique présentée au chapitre 5, puis des développements réalisés au chapitre 6, consacré à la réalisation et à l’implémentation de la solution proposée.

## 5 Méthodologie et planification

Dans le cadre de notre stage, nous avons adopté une démarche méthodique combinant **rétro-ingénierie incrémentale non destructive** et **développement logiciel itératif mock-first**. Cette approche répond directement à l'absence de documentation constructeur et à la disponibilité limitée du robot physique au laboratoire HESTIM.

Ce chapitre détaille notre mode de travail, la planification retenue, les outils employés, les premières découvertes sur le robot, la documentation produite, les apports du projet ainsi que les difficultés rencontrées durant la phase d'investigation.

### 5.1 Mode de travail et organisation

Le stage a réuni deux stagiaires aux **périmètres distincts** : la plateforme CYBEL (contenu technique du présent rapport, réalisée par Claudia LUSAMOTE KIMFUTA sur les branches `main`, `face presence` et `promote kisok poi main`) et un module **chatbot** développé en parallèle par Ghasane KHALIFI sur la branche `chatbot-cybel-projet-2026`, hors périmètre des chapitres de réalisation.

Pour la partie CYBEL, le mode de travail a reposé sur une démarche **incrémentale et autonome** : exploration documentaire, scripts Python de validation dans le dépôt `cybel`, développement mock-first puis tests terrain lorsque le robot était disponible.

Notre encadrante pédagogique, Mme MESKINI, notre encadrant en entreprise, M. TULA, ainsi que les membres du laboratoire nous ont accompagnés par des réunions régulières et des comptes rendus journaliers et hebdomadaires. Pour structurer l'avancement, nous avons maintenu un planning détaillé et centralisé nos hypothèses, blocages et solutions dans le dépôt Github du projet.

#### 5.1.1 Démarche de découverte protocolaire

Chaque hypothèse sur le protocole du robot a suivi le cycle suivant :

1. **Observation passive** écoute MQTT et introspection `rosbridge` avant toute publication ;
2. **Test isolé** script Python dédié dans `scripts/` ;
3. **Vérification de l'effet réel** une commande acceptée par `rosbridge` n'est pas forcément exécutée sur le châssis ;
4. **Intégration SDK** portage dans `MockRobot` puis `RealRobot` ;

5. **Documentation** mise à jour de docs/ et des constantes dans `sdk/constants.py`.

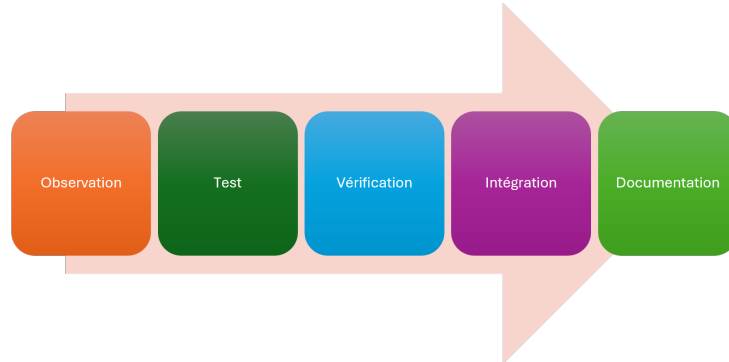


FIGURE 9 – Démarche de découverte

Les principes directeurs ont été : simuler d’abord, sécuriser le matériel (E-Stop accessible, pas de commandes destructives), et documenter au fil du temps chaque découverte reproductible.

### 5.1.2 Organisation hebdomadaire type

TABLE 10 – Organisation hebdomadaire indicative du projet

| Jours              | Activité principale                                       | Environnement       |
|--------------------|-----------------------------------------------------------|---------------------|
| Lundi–<br>Mercredi | Développement mock,<br>documentation, tests<br>unitaires  | PC, sans robot      |
| Jeudi              | Tests sur robot réel (si<br>disponible)                   | Wi-Fi TY1251D-03195 |
| Vendredi           | Intégration, déploiement<br>tablette Termux,<br>rédaction | PC + tablette       |

## 5.2 Planning du projet

Afin d’assurer une progression cohérente tout au long du projet et de respecter les objectifs fixés, un planning prévisionnel a été établi dès le début sous la forme d’un diagramme de Gantt. Celui-ci a permis de répartir les différentes activités, d’identifier les dépendances entre les tâches et de suivre l’avancement.

Le déroulement a été organisé en quatre grandes phases successives qui sont :



1. La première phase, consacrée à la **connectivité**, avait pour objectif de comprendre l'architecture matérielle et réseau du robot : analyse de la topologie, identification des équipements et balayage des ports ouverts.
2. La deuxième phase concernait l'**investigation du système**. Des travaux de rétro-ingénierie ont permis d'identifier les mécanismes de communication : exploration `rosapi`, analyse MQTT passive, étude du TTS via ADB et développement du pont Android `CybelTTSBridge`.
3. La troisième phase correspond au **développement de la plateforme CYBEL** : SDK Python (mock et réel), backend FastAPI avec WebSocket, interface opérateur, moteur de visite guidée (`TourEngine`), kiosque visiteur et déploiement Termux sur la tablette.
4. Enfin, la quatrième phase est dédiée à la **validation du système** : essais de navigation terrain, ajustement des coordonnées du parcours `lab_tour.json`, tests d'intégration et rédaction du présent rapport.

La Figure 10 présente le planning global du projet.

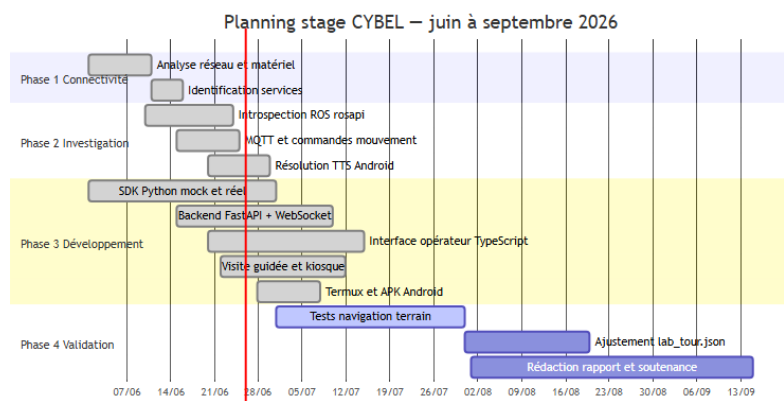


FIGURE 10 – Diagramme de Gantt du projet CYBEL

Plusieurs activités ont été menées en parallèle durant la phase de développement, ce qui a permis d'avancer sur l'interface web et le backend lorsque le robot n'était pas accessible.

TABLE 11 – Jalons principaux atteints (fin juin 2026)

| Jalon                                          | Période         | Statut  |
|------------------------------------------------|-----------------|---------|
| Connectivité <i>rosbridge</i> depuis le PC     | début juin 2026 | Atteint |
| TTS fonctionnel (ADB + <b>CybelTTSBridge</b> ) | mi-juin 2026    | Atteint |
| Interface opérateur complète                   | fin juin 2026   | Atteint |
| Kiosque affiché sur tablette Termux            | fin juin 2026   | Atteint |
| Parcours huit arrêts configuré                 | fin juin 2026   | Atteint |
| Visite guidée complète sans erreur 604         | juillet 2026    | Atteint |

## 5.3 Outils, langages et technologies utilisés

### 5.3.1 ROS (Robot Operating System)

Le châssis du robot exécute ROS pour la localisation SLAM, la planification (`move_base`) et l'exécution des trajectoires. CYBEL n'accède pas à ROS natif (C++/Python ROS) mais via *rosbridge* pont WebSocket JSON sur le port 9090.

TABLE 12 – Éléments ROS principaux utilisés par CYBEL

| Catégorie       | Exemples                                                                                                                |
|-----------------|-------------------------------------------------------------------------------------------------------------------------|
| Topics lecture  | <code>/robot_pose</code> , <code>/robot_status</code> , <code>/scan_filter</code> , <code>/detected_people_array</code> |
| Topics commande | <code>/mobile_base/commands/velocity</code> , <code>/navi_goal</code> , <code>/path_follower/cancel</code>              |
| Services        | <code>/poi</code> , <code>/change_location_mode</code> , <code>/global_localization</code>                              |
| Introspection   | <code>/rosapi/topics</code> , <code>/rosapi/services</code> , <code>/rosapi/subscribers</code>                          |

Les codes `nav_status` documentés sont : 600 (initialisation), 601 (prêt), 602 (en navigation), 603 (arrivé), 604 (erreur).



FIGURE 11 – Logo de ROS

### 5.3.2 FastAPI

FastAPI constitue le backend principal sur poste opérateur : API REST, WebSocket `/ws/telemetry`, routers (`robot`, `navigation`, `tour`, `speech`, etc.) et configuration via `pydantic-settings` (`ROBOT MOCK`, `ROBOT_HOST`). Sur tablette Termux, **Starlette** (`cybel_lite.py`) remplace FastAPI car `pydantic-core` ne compile pas sur Python 3.13 Termux.



FIGURE 12 – Logo de FastAPI

### 5.3.3 Python

Python a constitué le langage de toutes nos découvertes et un très puissant outils avec ces multiples bibliothèques nous avons pu concevoir le backend, les scripts, les SDK etc...



FIGURE 13 – Logo de Python

### 5.3.4 TypeScript et Vite

Deux applications web sans framework lourd que l'on a utilisé pour les interfaces :

- `frontend/` (port 5173) interface opérateur ;
- `frontend-kiosk/` (port 5174) kiosque visiteur bilingue FR/EN.

Le kiosque est compilé en bundle **IIFE** pour compatibilité WebView Chrome 49 (Android 7.1).



(a) TypeScript



(b) Vite

TABLE 13 – Logo de TypeScript et Vite

### 5.3.5 Android et Java

Deux APK Java compilées sans Gradle (java, d8, apksigner) :

- CybelTTSBridgesynthèse vocale via TextToSpeech ;
- CybelVisitorKioskWebView plein écran vers /kiosk/.



(a) Android



(b) Java

TABLE 14 – Logo de Android et Java

### 5.3.6 ADB (Android Debug Bridge)

ADB permet de déclencher le TTS depuis le PC développeur vers la tête Android (câble USB requis sur ce modèle). Sur tablette autonome, le broadcast local remplace ADB.



FIGURE 14 – Logo de ADB

### 5.3.7 rosbridge et communication réseau

TABLE 15 – Canaux de communication du projet

| Canal            | Adresse typique                | Protocole                 | Usage                   |
|------------------|--------------------------------|---------------------------|-------------------------|
| <i>rosbridge</i> | 10.42.0.1:9090 (PC)            | WebSocket<br>JSON         | Principal               |
| <i>rosbridge</i> | 192.168.20.22:9090<br>(Termux) | WebSocket<br>JSON         | Tablette                |
| MQTT             | 10.42.0.1:1883                 | MQTT 3.1.1                | Télémétrie pas-<br>sive |
| ADB              | USB                            | Android De-<br>bug Bridge | TTS depuis PC           |
| HTTP Ter-<br>mux | IP tablette:8000               | HTTP                      | Kiosque embar-<br>qué   |

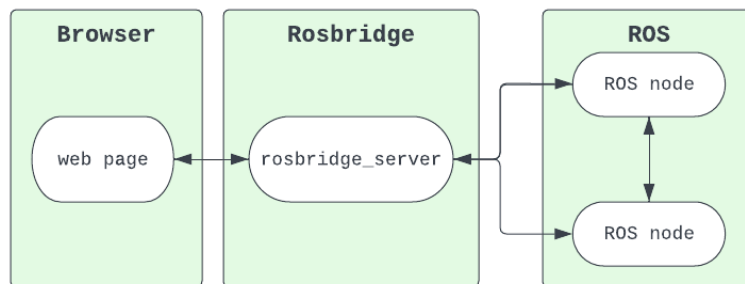


FIGURE 15 – Utilisation de rosbridge

## 5.4 Découverte du Robot CIOT TY1251D-03195 « CYBEL »

La phase initiale a permis d'identifier le matériel et les services réseau exposés sans accès shell au châssis (SSH verrouillé).

TABLE 16 – Services réseau découverts sur le robot

| Port | Service                            |
|------|------------------------------------|
| 9090 | <i>rosbridge</i> WebSocket         |
| 1883 | Broker MQTT                        |
| 8082 | Interface déploiement constructeur |
| 8088 | Interface debug CSST               |
| 21   | FTP                                |
| 22   | SSH (accès refusé)                 |

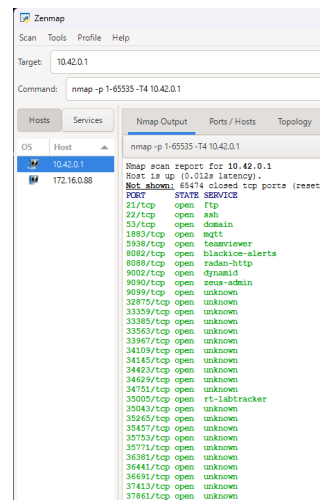


FIGURE 16 – Identification de des services réseau avec Zenmmmap

La topologie réseau s'est révélée dual-stack : le châssis est joignable en 10.42.0.1 depuis le Wi-Fi du robot, tandis que la tablette Android utilise un segment DHCP 172.16.0.0/16 et une liaison interne `eth0` sur 192.168.20.0/24. Depuis Termux, *rosbridge* est accessible sur 192.168.20.22:9090 et non sur 10.42.0.1.

#### 5.4.1 Zenmmmap

Zennmap (figure 16) est l'interface graphique officielle de Nmap, conçue pour rendre l'exploration et l'audit des réseaux plus accessibles. Elle a permis de lancer des scans sans retenir toutes les options complexes de la ligne de commande, en offrant une présentation claire des résultats sous forme de tableaux et graphiques. Grâce à ses profils de scan enregistrables, Zenmmmap facilite la répétition de tâches courantes et constitue un outil pratique pour les administrateurs réseau, les étudiants et les professionnels de la cybersécurité.



FIGURE 17 – Logo de nmap

### 5.4.2 JADX

JADX est un outil open source qui a permis de désassembler et décompiler les fichiers APK du constructeur (issus de leurs applications Android) en code Java lisible. Il nous a été particulièrement utile pour le développement, la recherche et pour analyser la structure interne des applications du constructeur, comprendre son fonctionnement et vérifier les comportements des APK. Grâce à son interface graphique et ses options en ligne de commande, JADX a facilité l'exploration du bytecode Android et la conversion en code source plus compréhensible.



FIGURE 18 – Logo de JADX

### 5.4.3 Termux

Termux est une application Android qui fournit un terminal Linux complet directement sur smartphone ou tablette. Elle permet d'exécuter des commandes Unix, d'installer des paquets via son propre gestionnaire, et d'accéder à des outils de développement ou de sécurité comme Python, Git ou SSH. Grâce à son environnement minimaliste mais puissant, Termux est utilisé aussi bien pour l'apprentissage de Linux que pour l'administration système ou le prototypage rapide sur mobile.

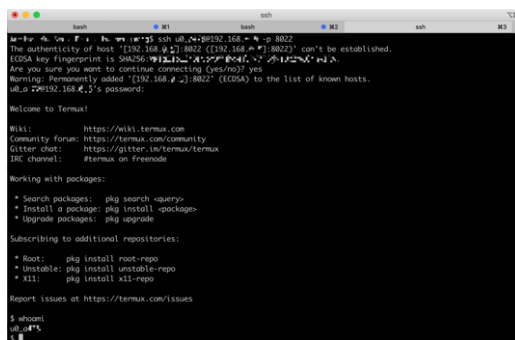


FIGURE 19 – Interface Termux

## 5.5 Proposition de documentation technique

Au fil du projet, nous avons structuré une documentation technique dans le répertoire `docs/` du dépôt, complétée par des scripts reproductibles dans `scripts/`. Les guides principaux sont :

- `ROBOT_CONNECTION.md` topologie réseau et paramètres de connexion ;
- `INTERFACE.md` interface opérateur ;
- `TTS_BRIDGE.md` pont de synthèse vocale Android ;
- `VISITOR_KIOSK.md` et `TERMUX_DEPLOY.md` kiosque et déploiement embarqué ;
- `labo/TERRAIN.md` procédure de validation terrain.

Les diagrammes Mermaid (architecture, séquences, cas d'utilisation) sont versionnés dans `docs/Sujet de stage/diagrammes/` et exportés en PNG pour le rapport. Cette documentation constitue la référence opérationnelle pour toute reprise du projet au laboratoire.

## 5.6 Apports du projet

Le projet CYBEL apporte à HESTIM une **plateforme tierce de commande et d'interaction**, indépendante de l'application propriétaire du constructeur. Les apports concrets sont :

- un protocole de communication ROS/*rosbridge* documenté et intégré dans un SDK Python testable en mode simulé ;
- une interface opérateur web (carte SLAM, LiDAR, téléopération, visite guidée, accueil vocal) ;
- un kiosque visiteur bilingue déployable sur la tablette du robot ;
- un canal TTS Android (*CybelTTSBridge*) absent du protocole ROS ;
- une base de connaissances laboratoire (*knowledgeV2-lab.json*) transformée en parcours navigable (*lab\_tour.json*).



## 5.7 Difficultés rencontrées durant la phase d'investigation

TABLE 17 – Synthèse des difficultés et solutions (phase d'investigation)

| Domaine       | Difficulté                                                         | Solution retenue                                                   |
|---------------|--------------------------------------------------------------------|--------------------------------------------------------------------|
| Réseau        | Topologie multi-IP, DHCP instable sur la tête Android              | Documentation<br><code>ROBOT_HOST</code> distinct<br>PC/Termux     |
| Réseau        | Latence Wi-Fi variable, timeouts <i>rosbridge</i>                  | Timeout 20 s, 3 tentatives, pas de <code>-reload</code> par défaut |
| Protocole     | Commande publiée sans abonné ROS                                   | Vérification via<br><code>/rosapi/subscribers</code>               |
| Protocole     | Mode auto<br>( <code>control_state:</code><br>30) bloque la téléop | Service<br><code>/change_location_mode</code><br>avant navigation  |
| Protocole     | Annulation navigation incomplète                                   | <code>/poi stop</code> + cancel service + arrêt vitesses           |
| TTS           | Aucun topic ROS pour la parole                                     | Application<br><code>CybelTTSBridge</code> + ADB                   |
| Système fermé | SSH verrouillé, app constructeur boîte noire                       | Rétro-ingénierie non destructive uniquement                        |
| Cartographie  | Une carte du laboratoire incorrecte                                | Réconstruction de la Carte du laboratoire                          |

## 5.8 Conclusion

La méthodologie adoptée observation, vérification, simulation puis intégration s'est avérée indispensable face à un système fermé et partiellement documenté. Le planning en quatre phases a été globalement respecté, avec une avance sur le déploiement tablette et un retard acceptable sur la validation navigation complète du parcours laboratoire. Les outils retenus (Python/FastAPI, TypeScript/Vite, *rosbridge*, ADB, Termux) forment un écosystème cohérent pour un stage à contraintes matérielles fortes. Le chapitre suivant détaille la réalisation concrète de cette architecture.

## 6 Réalisation et implémentation

Ce chapitre présente la mise en œuvre concrète de la plateforme CYBEL : environnement de développement, travaux de rétro-ingénierie, développement du backend, des applications Android et des interfaces web, intégration du TTS et assemblage des modules en un système opérationnel.

### 6.1 Mise en place de l'environnement

#### 6.1.1 Poste de développement

L'environnement de développement repose sur un PC sous Windows 11 avec Python 3.13, Node.js LTS, l'Android SDK en ligne de commande (`adb`, `javac`, `d8`) et l'IDE Cursor. Les dépendances Python sont installées via `pip install -r backend/requirements.txt`; les frontends via `npm install` dans `frontend/` et `frontend-kiosk/`.

Le lancement unifié s'effectue par :

Listing 1 – Commandes pour lancer l'interface contrôleur

```
cd cybel
python scripts\dev.py
```

Ce script démarre le backend sur le port 8000, l'interface opérateur sur 5173 et le kiosque sur 5174. La configuration robot se trouve dans `backend/.env` (`ROBOT MOCK=false`, `ROBOT_HOST=10.42.0.1` pour les tests depuis le PC connecté au Wi-Fi du robot).

#### 6.1.2 Environnement tablette (Termux)

Sur la tablette Android du robot, Termux héberge une version allégée du backend (`cybel_lite.py`). Le bootstrap est assuré par `scripts/termux/bootstrap_lite.sh`, le déploiement par `python scripts/deploy_termux.py -lite-only` et le démarrage par `/cybel/scripts/termux/start_cybel.sh`.

### 6.2 Découverte et analyse du robot

Le robot CIOT TY1251D-03195 combine un châssis Linux/ROS (localisation, planification, exécution) et une tête Android 7.1 (RK3399, 2 Go RAM, écran tactile 15,6"). L'application constructeur d'accueil constitue une boîte noire sans API publique; le travail de rétro-ingénierie a donc porté sur les canaux réseau exposés (*rosbridge*, MQTT, ADB).

L'analyse a confirmé que le châssis publie une télémétrie riche (pose, statut, LiDAR filtré, personnes détectées) et accepte des commandes de vitesse,

de navigation par coordonnées (`/navi_goal`) et par points d'intérêt (service `/poi`), sous réserve du mode de contrôle approprié.

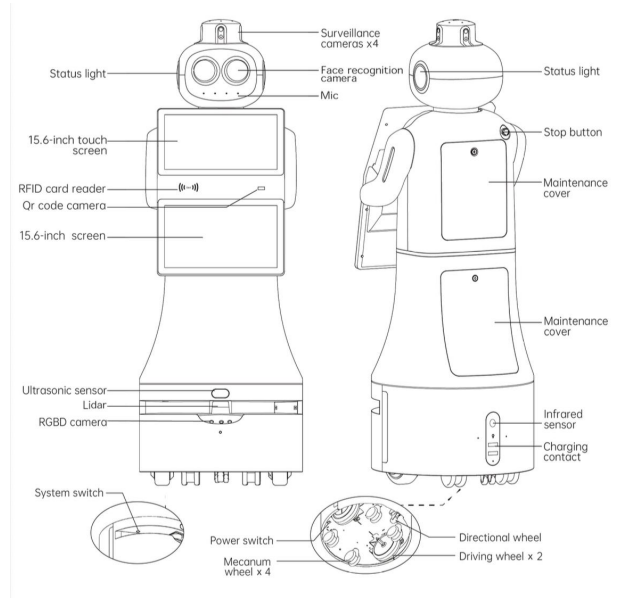


FIGURE 20 – Structure du robot

## 6.3 Reverse Engineering du système

### 6.3.1 Analyse réseau

La cartographie réseau a révélé trois segments interconnectés : le Wi-Fi AP du robot (10.42.0.0/24), le DHCP de la tête Android (172.16.0.0/16) et la liaison interne `eth0` (192.168.20.0/24). Cette découverte explique pourquoi une même commande peut fonctionner depuis le PC mais échouer depuis Termux si `ROBOT_HOST` n'est pas adapté.

### 6.3.2 Identification des protocoles

Quatre protocoles ont été identifiés et hiérarchisés :

1. **rosbridge v2** canal principal (JSON sur WebSocket) ;
2. **MQTT** télémétrie odométrique, observation passive ;
3. **HTTP** interfaces constructeur (non exploitées par CYBEL) ;
4. **ADB** accès à la tête Android pour le TTS.

### 6.3.3 Étude des commandes

Le tableau 18 synthétise les mécanismes validés sur matériel réel.

TABLE 18 – Commandes validées sur le robot CIOT

| Action                 | Mécanisme                                                                       |
|------------------------|---------------------------------------------------------------------------------|
| Téléopération          | Publication <code>/mobile_base/commands/velocity</code><br>(mode manuel requis) |
| Navigation point nommé | Service <code>/poi, command: go</code>                                          |
| Navigation coordonnée  | Publication <code>/navi_goal</code>                                             |
| Annuler navigation     | <code>/poi stop + /path_follower/cancel</code> + arrêt des vitesses             |
| Mode manuel/auto       | Service <code>/change_location_mode, mode: 0/1</code>                           |
| Relocalisation         | Service <code>/global_localization</code>                                       |
| Parole                 | Broadcast <code>com.cybel.ttsbridge.SPEAK</code> (hors ROS)                     |

## 6.4 Développement du backend CYBEL

Le backend suit l'architecture en couches définie au chapitre 4. Le module `sdk/` encapsule la communication *rosbridge* (`rosbridge.py`), l'implémentation réelle (`real_robot.py`) et simulée (`mock_robot.py`). Le service `robot_service.py` expose une interface unique aux routers FastAPI.

Les domaines API couverts sont : statut et mouvement du robot, navigation (`/api/navigation/goto`, `/goto-coordinate`, `/cancel`), carte SLAM, visite guidée (`/api/tour/start`, CRUD des arrêts), parole et réception. Le WebSocket `/ws/telemetry` diffuse pose, statut, carte et nuage LiDAR en temps réel.

Le client *rosbridge* implémente une connexion avec timeout de 20 s, trois tentatives, keepalive et reconnexion automatique en cas de silence prolongé. Le moteur **TourEngine** orchestre la visite : navigation séquentielle vers chaque arrêt de `data/lab_tour.json`, annonces vocales et gestion des erreurs (`nav_status 604`).

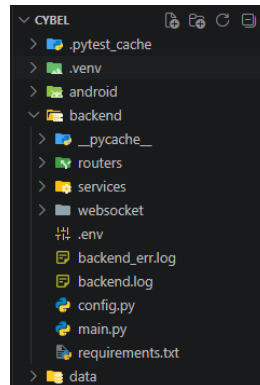


FIGURE 21 – architecture Backend

## 6.5 Développement de l'application Android

### 6.5.1 CybelTTSBridge

Application légère installée sur la tête Android du robot. Un `BroadcastReceiver` écoute l'action `com.cybel.ttsbridge.SPEAK`, initialise `TextToSpeech` (moteur Google TTS) et gère une file d'attente (`pendingText`) pour les messages reçus avant l'initialisation du moteur vocal.

### 6.5.2 CybelVisitorKiosk

Application `WebView` plein écran chargeant `http://<ip-tablette>:8000/kiosk/`. Un `BootReceiver` permet le démarrage automatique optionnel. Des correctifs `viewport-fit=cover` et `network_security_config` assurent l'affichage sur Android 7.1 et l'accès HTTP local à Termux.

### 6.5.3 CybelVisitorKiosk

Application `WebView` plein écran chargeant `http://<ip-tablette>:8001/kiosktest/`. Un `BootReceiver` permet le démarrage automatique optionnel. Des correctifs `viewport-fit=cover` et `network_security_config` assurent l'affichage sur Android 7.1 et l'accès HTTP local à Termux. Cette application est la version utiliser pour tester l'approche POI, en récupérant les coordonnées des différents appareils depuis l'application du constructeur "**Deployment tool**".

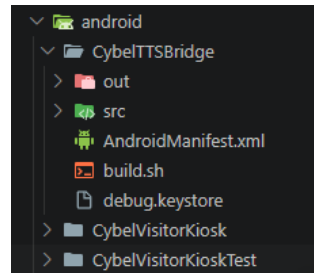


FIGURE 22 – architecture Android

## 6.6 Développement de l'interface Web

### 6.6.1 Interface opérateur (frontend/)

L'interface opérateur regroupe :

- une carte SLAM avec grille, LiDAR, pose, objectif et visiteurs détectés ;
- un panneau Points pour la navigation et la gestion des points locaux ;
- une téléopération (D-pad, bascule mode manuel, arrêt) ;
- une page Visite pour le CRUD de `lab_tour.json` et le suivi d'état ;
- une page Accueil pour les actions vocales et la FAQ.

### 6.6.2 Interface visiteur (frontend-kiosk/)

Le kiosque propose un parcours en trois écrans (accueil, visite en cours, fin ou erreur), une bascule FR/EN et un polling de `/api/tour/status` pendant la visite. La visite guidée autonome du laboratoire constitue le scénario principal : huit arrêts synchronisés depuis `knowledgeV2-lab.json`.

## 6.7 Mise en place du système TTS

La synthèse vocale ne transite pas par ROS. Le flux est le suivant :

1. le backend appelle `sdk/speech.py` ;
2. depuis le PC : subprocess ADB exécutant `am broadcast` vers `CybelTTSBridge` ;
3. depuis la tablette : `speak_local()` dans `cybel_lite.py` sans dépendance au PC.

Ce choix architectural a été imposé par l'absence de tout canal réseau ou ROS pour déclencher la parole sur la tête Android.

## 6.8 Intégration des différents modules

L'intégration bout en bout de la visite guidée suit la chaîne : kiosque tablette :

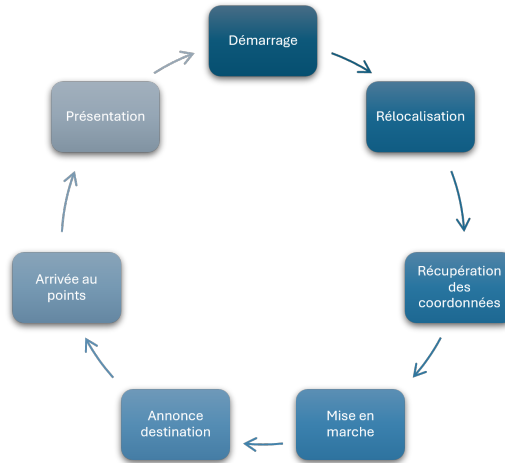


FIGURE 23 – Etape de la visite guidée

Le script `scripts/deploy_termux.py` automatise le pipeline de déploiement : build du kiosque, upload SFTP, bootstrap Termux et redémarrage de `cybel_lite`. Le script `preflight_lab0.ps1` vérifie avant chaque session terrain la connectivité, la santé de l'API et la présence du TTS.

## 6.9 Conclusion

L'implémentation CYBEL couvre l'intégralité de la chaîne du protocole robot aux interfaces utilisateur, en passant par une API structurée et un déploiement embarqué sur tablette. L'architecture en couches (SDK / API / UI) a absorbé les évolutions (visite guidée, TTS, Termux) sans remise en cause du cœur logiciel. Le principal travail restant porte sur l'alignement des coordonnées de `lab_tour.json` avec la carte SLAM réelle du laboratoire, objet du chapitre 7.

## 7 Validation, résultats et analyse critique

Ce chapitre présente la stratégie de validation adoptée, les scénarios de tests exécutés, les résultats obtenus, les indicateurs de performance observés, nos contributions, les difficultés résolues, les limites du projet et les perspectives d'amélioration.

### 7.1 Stratégie de validation

La validation s'organise sur quatre niveaux complémentaires :

TABLE 19 – Niveaux de validation du projet CYBEL

| Niveau             | Méthode                      | Périmètre                                        |
|--------------------|------------------------------|--------------------------------------------------|
| Unitaire           | <code>pytest</code>          | SDK, utils, tour, <i>rosbridge</i> , persistance |
| Intégration mock   | <code>ROBOT MOCK=true</code> | API + frontends sans robot                       |
| Intégration réelle | Tests manuels terrain        | Navigation, TTS, télémétrie                      |
| Acceptation        | Visite complète 8 arrêts     | Tablette + laboratoire FabLab                    |

Les critères de succès de la visite guidée sont : (1) démarrage depuis le kiosque sans erreur ; (2) atteinte de chaque coordonnée (`nav_status` → 603) ; (3) annonces vocales audibles ; (4) arrêt visiteur et E-Stop opérateur fonctionnels.

### 7.2 Scénarios de tests

Le tableau 20 recense les scénarios principaux et leur statut à fin juin 2026.



TABLE 20 – Scénarios de tests et résultats

| ID  | Scénario                               | Résultat attendu                            | Statut        |
|-----|----------------------------------------|---------------------------------------------|---------------|
| T1  | Ping +<br><code>robot_status.py</code> | Messages JSON<br><code>/robot_status</code> | OK            |
| T2  | Téléopération mode<br>manuel           | Déplacement au D-<br>pad                    | OK            |
| T3  | Navigation vers point<br>carte         | <code>nav_status</code><br>602→603          | OK            |
| T4  | Annulation navigation                  | Arrêt + disparition<br>trajectoire          | OK            |
| T5  | TTS via ADB                            | Audio audible                               | OK            |
| T6  | TTS tablette (broad-<br>cast)          | Audio sans PC                               | OK            |
| T7  | Kiosque WebView                        | Écran d'accueil CY-<br>BEL                  | OK            |
| T8  | Visite 9 arrêts complète               | Sans erreur 604                             | OK            |
| T9  | Relâcher E-Stop sans<br>reprise nav    | Robot immobile                              | OK            |
| T10 | <code>pytest tests/</code>             | Tous les tests passés                       | OK (78 tests) |

## 7.3 Résultats obtenus

### 7.3.1 Fonctionnalités livrées

À ce stade du projet, les livrables suivants sont opérationnels en démonstration :

- plateforme opérateur complète (carte, LiDAR, visiteurs, visite, accueil) ;
- kiosque visiteur bilingue déployé sur tablette Termux ;
- protocole ROS documenté dans `sdk/constants.py` ;
- pont TTS Android (CybelTTSBridge) ;
- parcours laboratoire neuf arrêts dans `data/lab_tour.json`.

### 7.3.2 Incidents terrain documentés

TABLE 21 – Incidents terrain et résolutions

| Incident                           | Cause                                                      | Résolution                            |
|------------------------------------|------------------------------------------------------------|---------------------------------------|
| Erreur 604 en visite               | Coordonnée dans obstacle ou chemin bloqué                  | Ajustement <code>lab_tour.json</code> |
| Timeout handshake <i>rosbridge</i> | Wi-Fi saturé                                               | Retries, pas de reload                |
| Objectif conservé après annulation | Télémétrie non effacée                                     | Masquage + <code>/poi stop</code>     |
| Bouton Arrêt kiosque inefficace    | Appel <code>haltTour</code> au lieu de <code>cancel</code> | Correction frontend                   |

## 7.4 Indicateurs de performance

Le tableau 22 présente les indicateurs mesurés en conditions réelles et en développement local.

TABLE 22 – Indicateurs de performance observés

| Indicateur                           | Valeur observée         | Commentaire                               |
|--------------------------------------|-------------------------|-------------------------------------------|
| Temps de réponse API REST            | < 100 ms                | Localhost PC, hors <i>rosbridge</i>       |
| Connexion <i>rosbridge</i>           | 2–60 s                  | Selon qualité Wi-Fi ; 3× timeout 20 s max |
| Fiabilité connexion <i>rosbridge</i> | ~90 %                   | Session stable, dépend du Wi-Fi           |
| Stabilité communications             | Reconnexion auto        | Watchdog après 25 s de silence            |
| Fréquence télémétrie pose            | ~10 Hz                  | Topic <code>/robot_pose</code>            |
| Fréquence LiDAR                      | ~25 Hz                  | Topic <code>/scan_filter</code>           |
| Latence Wi-Fi (ping robot)           | 89-91654 ms             | Très variable                             |
| Robustesse navigation                | Erreurs 604 ponctuelles | Coordonnées à calibrer terrain            |
| Tests unitaires                      | 78/78 passés            | Juin 2026                                 |

## 7.5 Contributions personnelles

Le présent rapport documente la plateforme CYBEL. Les contributions des deux stagiaires sont distinctes et complémentaires.

**Claudia LUSAMOTE - plateforme CYBEL (branche main).** L'ensemble des travaux décrits dans ce rapport relève de sa contribution :

1. rétro-ingénierie du protocole de communication du CIOT TY1251D ;
2. conception et développement de l'architecture CYBEL (SDK, API FastAPI, interfaces web TypeScript/Vite) ;
3. résolution du canal TTS (`CybelTTSBridge`, ADB/broadcast) ;
4. déploiement embarqué Termux et kiosque visiteur ;
5. conception du parcours laboratoire (`lab_tour.json`) ;
6. documentation technique (`docs/`, scripts, diagrammes) ;
7. corrections itératives terrain (navigation, *rosbridge*, erreurs 604).

**Ghassane KHALIFI - module chatbot (branche chatbot-cybel-projet-2026).**

Il a développé, sur une branche séparée du dépôt GitHub du projet, un module **chatbot** d'interaction conversationnelle pour le robot. Ce travail constitue une extension parallèle au périmètre CYBEL décrit dans les chapitres précédents et n'est pas intégré à la branche `main` à ce stade.

## 7.6 Difficultés rencontrées et solutions apportées

Au-delà des difficultés d'investigation (chapitre 5), la phase de développement et de validation a nécessité des correctifs spécifiques :

- **FastAPI sur Termux** : remplacement par Starlette (`cybel_lite.py`) ;
- **Écran blanc WebView** : build IIFE et correctifs `safe-area` ;
- **Saturations *rosbridge*** : `dev.py` sans `-reload` par défaut, instance unique du backend ;
- **Navigation 604** : affinage progressif des coordonnées sur carte SLAM réelle via sessions terrain documentées dans `docs/labo/TERRAIN.md`.

## 7.7 Limites du projet

1. Dépendance au Wi-Fi du robot, pas de contrôle hors réseau TY1251D-03195.
2. Un seul robot de test, généralisation à d'autres modèles CIOT non garantie.
3. Coordonnées du parcours, calibrage manuel sur carte SLAM.

4. Pas de ROS natif, dépendance totale à *rosbridge*.
5. Android 7.1, contraintes WebView, pas de modules ES modernes.
6. Backend lite Termux, sous-ensemble des fonctions opérateur.
7. Pas de CI/CD ni déploiement industrialisé.
8. Reconnaissance vocale visiteur, non implémentée en v1.

## 7.8 Perspectives d'amélioration

TABLE 23 – Perspectives d'amélioration priorisées

| Priorité | Amélioration                                                                            |
|----------|-----------------------------------------------------------------------------------------|
| Haute    | Calibrer les huit arrêts sur carte réelle (éliminer les erreurs 604)                    |
| Haute    | Connexion <i>rosbridge</i> non bloquante au démarrage FastAPI                           |
| Moyenne  | Aligner <code>cybel_lite.py</code> sur toute la logique de <code>real_robot.py</code>   |
| Moyenne  | Tests d'intégration <i>rosbridge</i> simulé ; boot auto Termux                          |
| Basse    | Synchronisation POI robot ↔ <code>lab_tour.json</code> ; reconnaissance vocale visiteur |

## 7.9 Conclusion

La validation démontre que CYBEL atteint son objectif principal : **commander et faire interagir le robot sans l'application propriétaire**. Les tests unitaires et d'intégration mock sont au vert ; les tests terrain confirment téléopération, TTS, télémétrie et navigation ponctuelle. La visite guidée complète sur huit arrêts reste le dernier jalon de validation, conditionnée par l'affinage des coordonnées et la stabilité réseau. Les limites identifiées sont connues et documentées ; les perspectives constituent une feuille de route pour la suite du projet au laboratoire HESTIM.

## CONCLUSION GÉNÉRALE

Le projet CYBEL, réalisé dans le cadre de notre stage à HESTIM, visait à concevoir une plateforme tierce de commande et d'interaction pour le robot de réception CIOT TY1251D-03195, en l'absence de toute documentation officielle du constructeur. Cette problématique doublement exigeante sur le plan technique (protocole inconnu) et organisationnel (robot peu disponible) a orienté l'ensemble de la démarche retenue.

La rétro-ingénierie incrémentale et non destructive du protocole ROS, menée via *rosbridge*, MQTT et ADB, a permis de reconstruire un modèle de communication fiable et documenté. Sur cette base, une architecture en couches a été livrée SDK Python, API FastAPI, interfaces web opérateur et visiteur, applications Android complémentaires validée en mode simulé et partiellement sur le terrain.

Les résultats obtenus confirment la faisabilité de l'objectif initial : une interface opérateur complète (carte SLAM, LiDAR, téléopération, gestion de visite), un kiosque visiteur bilingue déployé sur la tablette Termux, un canal de synthèse vocale opérationnel et un parcours pédagogique de neuf arrêts configuré. Les soixante-dix-huit tests unitaires automatisés et les scénarios terrain validés (T1 à T7, T9, T10) attestent de la robustesse logicielle ; la visite guidée complète sans interruption (scénario T8) est l'un jalons finaux, en cours de calibrage des coordonnées sur la carte réelle du laboratoire.

Ce travail démontre qu'un robot de service « fermé » peut être intégré dans un écosystème pédagogique personnalisé, à condition d'accepter une phase d'investigation protocolaire et de maintenir une discipline stricte de validation sur le matériel réel. Les compétences mobilisées réseaux, ROS/*rosbridge*, développement full-stack, Android embarqué, robotique mobile correspondent au profil visé par la filière Génie Informatique à HESTIM.

Les perspectives identifiées (calibrage terrain, parité backend lite/embarqué, tests d'intégration réseau simulés) constituent une feuille de route pour une exploitation durable de CYBEL au FabLab. Au-delà du livrable technique, le projet laisse une documentation reproductible (*docs/*, scripts, diagrammes) permettant à d'autres étudiants ou chercheurs de reprendre le travail sans repartir de zéro sur la rétro-ingénierie du robot.

## BIBLIOGRAPHIE

1. Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R., & Ng, A. Y. (2009). *ROS : an open-source Robot Operating System*. ICRA Workshop on Open Source Software.
2. Thrun, S., Burgard, W., & Fox, D. (2005). *Probabilistic Robotics*. MIT Press.
3. Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). *Introduction to Autonomous Mobile Robots*. MIT Press.
4. OASIS. (2014). *MQTT Version 3.1.1 OASIS Standard*. OASIS Open.
5. OWASP Foundation. *OWASP Internet of Things Top 10*. <https://owasp.org/www-project-internet-of-things/>
6. HESTIM Engineering & Business School. *Development of a Custom Interaction and Control System for an Android-Based Service Robot* document de sujet de stage (M. Sridath Tula, HESTIM).

## WEBOGRAPHIE

1. Robot Web Tools - rosbridge\_suite : [https://github.com/RobotWebTools/rosbridge\\_suite](https://github.com/RobotWebTools/rosbridge_suite)
2. Robot Web Tools - roslibjs : <https://github.com/RobotWebTools/roslibjs>
3. FastAPI - documentation officielle : <https://fastapi.tiangolo.com>
4. Vite - documentation officielle : <https://vitejs.dev>
5. Pydantic - documentation : <https://docs.pydantic.dev>
6. Mozilla Developer Network - Web Speech API : [https://developer.mozilla.org/en-US/docs/Web/API/Web\\_Speech\\_API](https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API)
7. Android Developers - TextToSpeech : <https://developer.android.com/reference/android/speech/tts/TextToSpeech>
8. Android Developers - Debug Bridge (ADB) : <https://developer.android.com/tools/adb>
9. Termux Wiki : <https://wiki.termux.com>
10. Mermaid - diagrammes as code : <https://mermaid.js.org>
11. Mermaid Live Editor (export PNG pour Overleaf) : <https://mermaid.live>
12. Dépôt GitHub du projet CYBEL : <https://github.com/Claude20022002/Projet-Stage-2026-Cybel>
13. Branche chatbot (G. Khalifi) : <https://github.com/Claude20022002/Projet-Stage-2026-Cybel/tree/chatbot-cybel-projet-2026>
14. Claude AI : <https://claude.ai/new>

## ANNEXES

### Annexe A - Captures d'écran

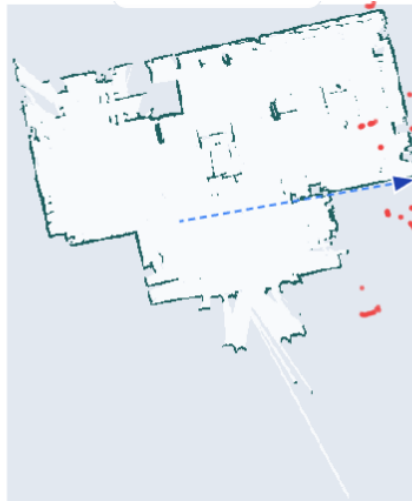


FIGURE 24 – Ancienne carte SLAM du laboratoire de l'interface opérateur

```
PS C:\Users\clusa> ssh -p 8022 u0_a92@172.16.0.132
PS C:\Users\clusa> ssh -p 8022 u0_a92@172.16.0.131
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@  WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!  @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle attack)!
It is also possible that a host key has just been changed.
The fingerprint for the ED25519 key sent by the remote host is
SHA256:2QrvZe29VR/PVLjRtQnhkI3oJL0m0avLd/FpGff97E.
Please contact your system administrator.
Add correct host key in C:\Users\clusa/.ssh/known_hosts to get rid of
this message.
Offending ED25519 key in C:\Users\clusa/.ssh/known_hosts:10
Host key for [172.16.0.131]:8022 has changed and you have requested stri
ct checking.
Host key verification failed.
PS C:\Users\clusa> ssh-keygen -R [172.16.0.132]:8022
# Host [172.16.0.132]:8022 found: line 11
# Host [172.16.0.132]:8022 found: line 12
# Host [172.16.0.132]:8022 found: line 13
C:\Users\clusa/.ssh/known_hosts updated.
Original contents retained as C:\Users\clusa/.ssh/known_hosts.old
```

FIGURE 25 – Terminal vérification de connectivité et génération de clé SSH



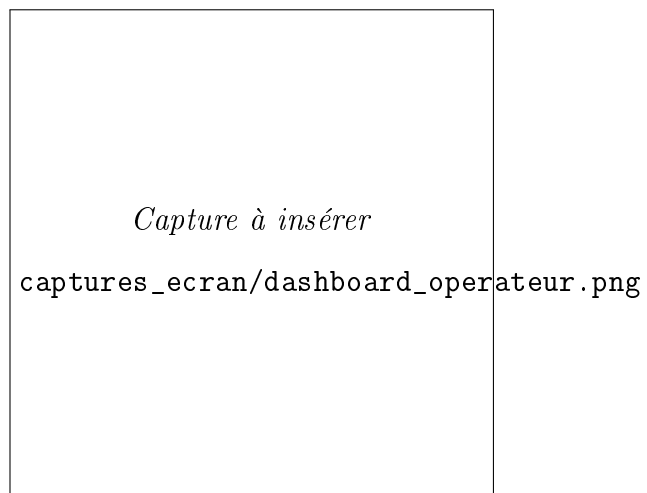


FIGURE 26 – Dashboard opérateur - mode robot réel (*à insérer*)

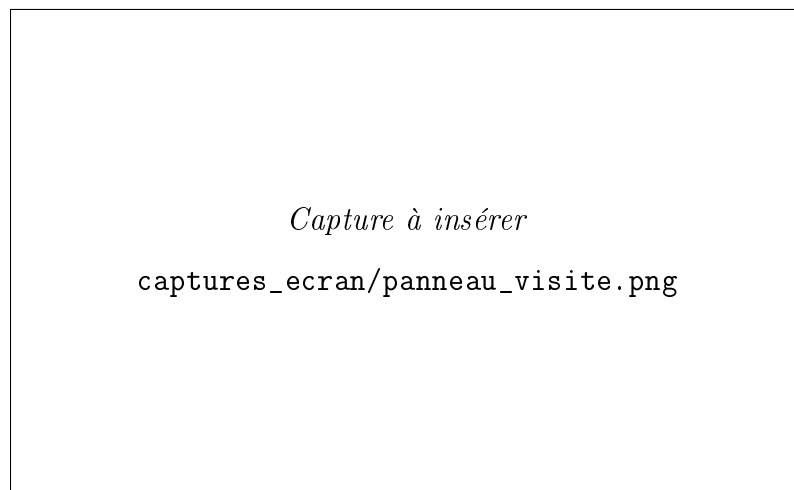


FIGURE 27 – Panneau visite guidée - page Visite (*à insérer*)

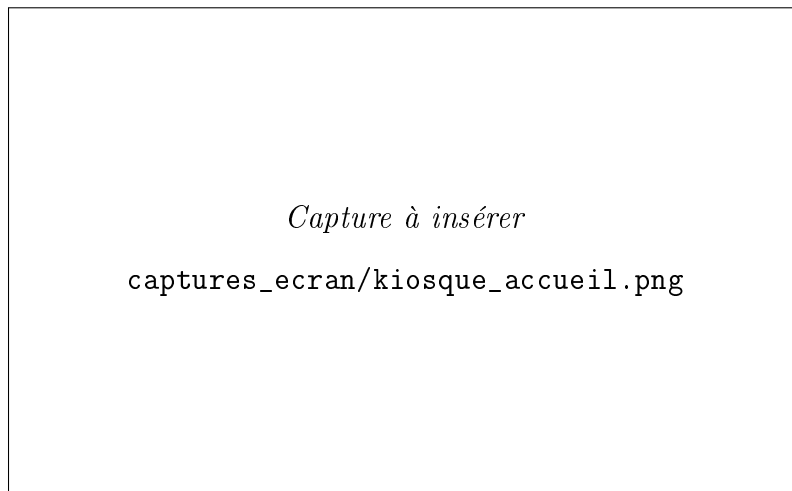


FIGURE 28 – Kiosque visiteur - écran d'accueil FR (*à insérer*)

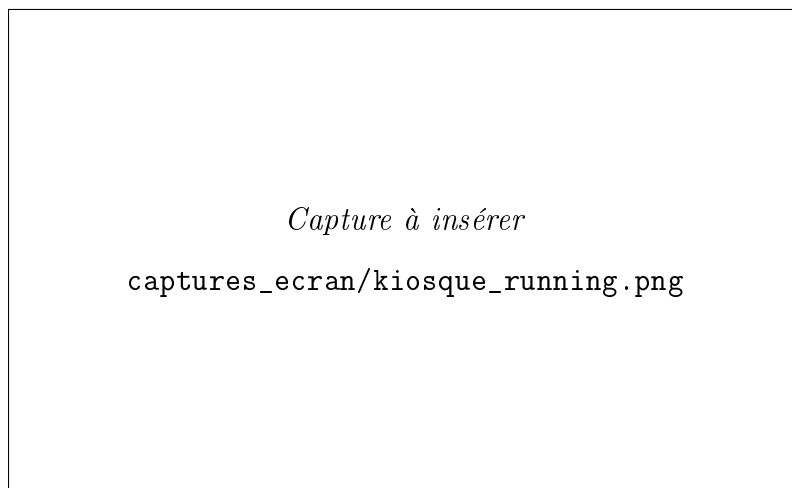


FIGURE 29 – Kiosque visiteur - visite en cours (*à insérer*)

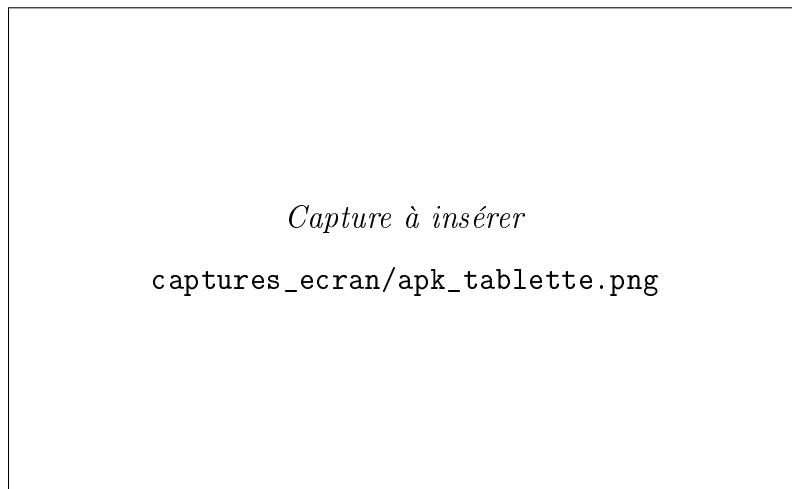


FIGURE 30 – Application CybelVisitorKiosk sur tablette (*à insérer*)

## Annexe B - Dépôt source du projet

- **Dépôt principal CYBEL** (branche main) :  
<https://github.com/Claude20022002/Projet-Stage-2026-Cybel>
- **Branche chatbot** (G. Khalifi) :  
<https://github.com/Claude20022002/Projet-Stage-2026-Cybel/tree/chatbot-cybel-projet-2026>

## Annexe C - Commandes utiles

Listing 2 – Commandes PC (PowerShell)

```
# Verification reseau
ping 10.42.0.1
python scripts\robot_status.py

# Lancement developpement
python scripts\dev.py

# Tests unitaires (78 tests)
python -m pytest tests/ -v

# TTS test (USB branche)
adb shell am broadcast -n com.cybel.ttsbridge/.
    SpeakReceiver ^
    -a com.cybel.ttsbridge.SPEAK --es text "Test HESTIM"
```

```
# Deploiement tablette
python scripts\deploy_termux.py --skip-kiosk-build --lite
    -only --host <IP>
```

Listing 3 – Commandes Termux (tablette)

```
cd ~/cybel/scripts/termux
./stop_cybel.sh && ./start_cybel.sh
curl http://127.0.0.1:8000/api/health
```

## Annexe D - Extraits de code significatifs

### D.1 - Annulation de navigation (sdk/real\_robot.py).

```
async def _cancel_navigation(self) -> None:
    for args in [{"command": "stop"}, ...]:
        await self._client.call_service(
            ROS_SERVICES["poi"], args, timeout=3.0)
        await call_service_first(self._client, ...)
        await self._publish_velocity(0.0, 0.0)
```

### D.2 - Publication d'un objectif de navigation.

```
await self._client.publish(ROS_TOPICS["navi_goal"], {
    "header": {"frame_id": "map"},
    "pose": {
        "position": {"x": x, "y": y, "z": 0.0},
        "orientation": {"z": sin(theta/2), "w": cos(theta/2)},
    },
})
```

### D.3 - Structure d'un arrêt (data/lab\_tour.json).

```
{
    "id": "cnc_router",
    "equipment_fr": "Routeur CNC",
    "x": -8.38, "y": 1.45, "theta": 0.88,
    "speech_fr": "Le routeur CNC permet d'usiner...",
    "dwell_seconds": 12
}
```

## Annexe E - Diagrammes détaillés

Sources Mermaid dans docs/Sujet de stage/diagrammes/ du dépôt CYBEL. Remplacer chaque cadre par `\includegraphics` après export PNG.

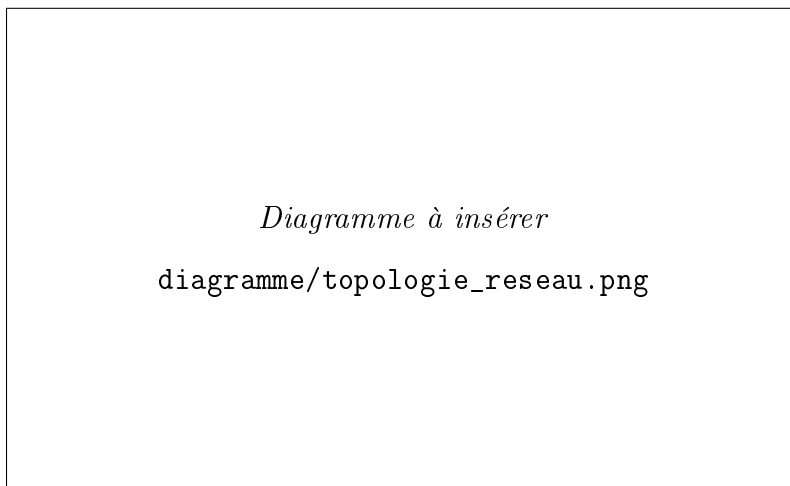


FIGURE 31 – Topologie réseau du robot (*à insérer*)

## Annexe F - Documentation technique complémentaire

TABLE 24 – Index de la documentation technique CYBEL

| Document                  | Chemin dans le dépôt     |
|---------------------------|--------------------------|
| README projet             | README.md                |
| Guide interface opérateur | docs/INTERFACE.md        |
| Pont TTS                  | docs/TTS_BRIDGE.md       |
| Kiosque visiteur          | docs/VISITOR_KIOSK.md    |
| Déploiement Termux        | docs/TERMUX_DEPLOY.md    |
| Connexion robot           | docs/ROBOT_CONNECTION.md |
| Procédure terrain         | docs/labo/TERRAIN.md     |